

- 1. Document Info
- 2. Purpose
 - New team reading guide (start here)
- 3. How to Read This Document
 - What is a tool
 - What is a library
 - What is a managed service
 - What is an external integration
 - Difference between open source, offline, free online, free trial, and paid
 - Open source
 - Offline
 - Free online
 - Free trial / free tier
 - Paid
 - How to interpret each tool entry
- 4. Cost Classification
 - 4.1 Free and Open Source
 - 4.2 Free Offline AI Tools
 - 4.3 Free Online AI Tools / APIs
 - 4.4 Free Tier / Free Trial
 - 4.5 Paid Now
 - 4.6 Paid Later at Scale
- 5. Usage Classification
 - 5.1 Development Tools
 - 5.2 Testing Tools
 - 5.3 AI / ML Tools
 - 5.4 Infrastructure / DevOps Tools
 - 5.5 Data and Storage Tools
 - 5.6 Monitoring and Security Tools
 - 5.7 External APIs and Integrations
 - 5.8 Productivity / Team Tools
- 6. Core Principles
 - Prefer approved stack first
 - Do not introduce new tools without review
 - Version pinning and version governance are mandatory
 - Upgrade governance and dependency ownership are mandatory
 - Prefer open-source for local development
 - Prefer free offline AI for experimentation, privacy, and backup workflows
 - Prefer free online AI only for low-risk prototyping
 - Move to paid tools only when justified by scale, security, reliability, or product quality
- 7. Inventory Summary Table
 - Table field meanings
 - Master inventory table (consolidated from Sections 8 through 17)
- 8. Free Offline AI Tools
 - 8.1 Local LLM Runtimes
 - Ollama
 - LM Studio
 - llama.cpp-compatible runtimes
 - 8.2 Local Open Models
 - Gemma family
 - Gemma 3
 - Gemma 4
 - Other practical open-model families
 - 8.3 Local Coding Assistants
 - Ollama with coding models
 - LM Studio local coding workflows

- Local code-oriented models
- 8.4 Local Embedding / RAG Tools
 - LM Studio document chat
 - pgvector in local PostgreSQL
 - Local embedding models
- 8.5 Local Speech and Audio Tools
 - Whisper
 - pyannote.audio
 - FFmpeg and local audio preprocessing tools
- 8.6 Recommended Offline Use Cases
- 9. Free Online AI Tools and APIs
 - 9.0 Quick reference (short form)
 - 9.1 Free Chat / Research Tools
 - Google AI Studio
 - Free research/chat tools from major providers
 - 9.2 Free AI APIs
 - Groq
 - Hugging Face hosted inference options
 - xAI / Grok API routes (evaluation only)
 - 9.3 Free Model Hubs
 - Hugging Face Hub
 - Kaggle model distribution path for Gemma
 - 9.4 Best Online Tools for Prototyping
 - 9.5 Risks and Limits of Free Online Tools
 - 9.6 Recommended Online Use Cases
- 10. Frontend Tools
 - 10.1 Frameworks
 - Next.js 15
 - TypeScript
 - 10.2 UI Libraries
 - Tailwind CSS
 - shadcn/ui
 - Radix UI
 - 10.3 State Management
 - TanStack Query
 - Zustand
 - Next.js router state
 - 10.4 Forms and Validation
 - React Hook Form
 - Zod
 - 10.5 Charts and Visualization
 - Recharts
 - Alternative rule for charts
 - 10.6 Frontend Testing
 - Playwright
 - Jest / component-level test utilities
 - 10.7 Specialized Frontend Utilities
 - Axios
 - Lucide React
 - Framer Motion
 - jsPDF and html2canvas
- 11. Backend Tools
 - 11.1 Core Frameworks
 - NestJS v11
 - TypeScript
 - Prisma ORM
 - SQLAlchemy and Alembic

- 11.2 API and Validation
 - REST API pattern
 - Zod
 - Swagger / API documentation style
- 11.3 Authentication and Authorization
 - Supabase Auth
 - JWT Guards
 - RBAC
- 11.4 Queue and Event Tools
 - BullMQ
 - Redis
 - Event contracts
- 11.5 Backend Testing
 - Jest
 - Supertest
 - Testcontainers
 - Backend testing principle
- 12. AI and ML Stack
 - 12.1 Primary LLM Providers
 - OpenAI API
 - OpenAI for structured JSON output
 - 12.2 Free / Backup LLM Providers
 - LiteLLM fallback routing
 - Anthropic fallback path
 - Ollama and local open-model fallback
 - Google AI Studio / Groq / Hugging Face hosted options
 - 12.3 AI Frameworks and Orchestration
 - Python 3.12
 - FastAPI
 - LiteLLM
 - LangGraph
 - BullMQ-driven async AI execution
 - 12.4 NLP Libraries
 - spaCy
 - Topic modeling and clustering utilities
 - Intent detection pipelines
 - 12.5 Speech-to-Text and Audio
 - Whisper
 - AssemblyAI
 - pyannote.audio
 - FFmpeg and preprocessing utilities
 - 12.6 Embeddings and Vector Search
 - pgvector
 - /v1/embed AI endpoint
 - sentence-transformers / local embedding models
 - 12.7 ML / Forecasting Libraries
 - scikit-learn
 - XGBoost
 - Statistical forecasting logic
 - 12.8 AI Testing and Evaluation Tools
 - Golden datasets
 - Confidence-score gating
 - Contract tests for AI endpoints
 - Human review workflows
 - 12.10 Specialized AI Tools and Libraries
 - LangChain
 - Groq (Llama 3.3 70B & 3.1 70B)

- Kokoro TTS
 - scikit-learn, XGBoost, and CatBoost
- 12.9 AI service operating depth (FastAPI, LiteLLM, LangGraph, spaCy, sentence-transformers)
 - FastAPI operating profile
 - LiteLLM operating profile
 - LangGraph operating profile
 - spaCy operating profile
 - sentence-transformers operating profile
- 13. Data Layer
 - 13.1 Primary Database
 - PostgreSQL
 - Shared PostgreSQL with Row-Level Security
 - Supabase PostgreSQL
 - 13.2 Cache and Queue Storage
 - Redis
 - BullMQ on Redis
 - Upstash Redis HA
 - 13.3 Search Engine
 - Meilisearch
 - 13.4 Analytics Store
 - ClickHouse
 - Analytics separation principle
 - 13.5 Object / File Storage
 - Supabase Storage or equivalent object storage
 - Audio retention and deletion policy
 - 13.6 Data Export / Warehouse Connectors
 - Data Cloud export layer
 - Snowflake connector
 - BigQuery connector
 - Databricks connector
 - Amazon S3 and Redshift connectors
 - Connector rule
- 14. Infrastructure and DevOps
 - 14.1 Local Development
 - Docker Compose
 - Local parity principle
 - 14.2 Containerization
 - Docker
 - Service-specific container boundaries
 - 14.3 Hosting
 - Railway
 - AWS ECS / Fargate
 - Hosting migration trigger
 - 14.4 CI/CD
 - GitHub Actions
 - PR quality gates
 - Dependency vulnerability scanning
 - 14.5 Secrets Management
 - Doppler
 - Secrets discipline
 - 14.7 Required API Keys and Credentials
 - 14.6 CDN / WAF / Edge Services
 - Cloudflare
 - Webhook protection use case
- 15. Observability and Security
 - 15.1 Error Tracking
 - Sentry

- Sentry sourcemaps and release visibility
- 15.2 Logging
 - Better Stack
 - Structured logging discipline
- 15.3 Monitoring and Alerting
 - Grafana
 - Better Stack uptime monitoring
 - Alerting on integration and webhook failures
- 15.4 Incident Management
 - Incident workflow maturity
 - PagerDuty or equivalent later
- 15.5 Security and Compliance Tools
 - Row-Level Security (RLS)
 - RBAC
 - Supabase Auth + JWT
 - Configure Compliance Settings
 - HMAC webhook verification
 - Audit and retention controls
- 16. External APIs and Integrations
 - 16.1 CRM Integrations
 - Salesforce
 - HubSpot
 - Microsoft Dynamics 365
 - CRM integration rule
 - 16.2 Meeting / Call Integrations
 - Zoom
 - Google Meet
 - Microsoft Teams
 - Dialers / telephony tools
 - 16.3 Email Integrations
 - Gmail
 - Outlook / Office 365
 - Email boundary rule
 - 16.4 Messaging / Collaboration Integrations
 - Slack
 - Collaboration integration rule
 - 16.5 Billing Integrations
 - Billing provider placeholder
 - Recommendation for billing tools
 - 16.6 Other Third-Party Services
 - OpenAI API
 - Whisper / AssemblyAI
 - LinkedIn Sales Navigator
 - Warehouse targets
- 17. Testing Stack
 - 17.1 Unit Testing
 - Jest
 - pytest
 - Unit testing rule
 - 17.2 Integration Testing
 - Supertest
 - Testcontainers
 - Prisma + Zod + RLS validation scripts
 - Integration testing rule
 - 17.3 End-to-End Testing
 - Playwright
 - Event-flow verification tests

- E2E testing rule
- 17.4 Load and Performance Testing
 - k6
 - Queue and AI pipeline performance tests
 - Performance testing rule
- 17.5 Test Data / Mocking / Containers
 - Mocking external services
 - Test fixtures and golden samples
 - Containers for realistic test environments
- 17.6 AI Evaluation and Prompt Testing
 - Golden datasets
 - Prompt versioning
 - Confidence-score gating
 - Human review and prompt review loops
- 18. Tool Approval Status
 - Copy-paste table
- 19. Payment Planning
 - Copy-paste table
 - Payment planning rule
 - Payment planning ownership
- 20. Risks and Notes
 - Vendor lock-in risks
 - API cost risks
 - Free-tier limitations
 - Privacy risks for free online tools
 - Migration risks
 - Security considerations
 - General notes
- 21. Libraries That Will Be Used
 - Frontend libraries
 - Backend libraries
 - AI and ML libraries
 - Data and search libraries / tools
 - DevOps and observability tools
 - Testing libraries
 - Integration SDKs and APIs
 - Important architecture note
- 23. Change Log
- 24. Executive Summary
 - Detailed summary table
 - Final summary note

1. Document Info

- **Document name:** Tooling and Services Inventory
- **Project:** R-Revenue Intelligence
- **Organization:** Relanto.ai
- **Owner:** Tech Lead / Engineering Lead
- **Contributors:** Backend Engineers, Frontend Engineers, AI/ML Engineers, QA Engineers, DevOps Engineers, Product Managers
- **Version:** 0.1
- **Status:** Draft
- **Last updated:** April 22, 2026
- **Next review date:** July 2026
- **Review cadence:** Every 3 months, or immediately after any major architecture decision, new integration, new AI model adoption, security change, or infrastructure change

- **Source references:** System Architecture Document (SAD), feature design documents, API docs, deployment configs, CI/CD configs, and procurement decisions

2. Purpose

This document is the single working inventory of all tools, libraries, services, APIs, models, platforms, and external integrations used or planned for the development, testing, deployment, monitoring, and scaling of the R-Revenue Intelligence platform. It exists so that anyone on the team can quickly understand what we use, why we use it, whether it is free or paid, whether it works offline or online, and where it fits in the product lifecycle. It also helps prevent random tool sprawl, duplicate purchases, and unsupported engineering decisions.

This document should be used by:

- **Backend engineers**, to know which frameworks, queues, databases, APIs, and supporting services are approved for server-side development and integrations.
- **Frontend engineers**, to know which UI frameworks, state libraries, charts, testing tools, and hosting services are approved for product development.
- **AI/ML engineers**, to know which LLM providers, local models, orchestration frameworks, transcription tools, vector tools, and evaluation utilities are approved for AI feature development.
- **QA engineers**, to know which test frameworks, browser automation tools, load testing tools, mocks, and test environments are expected for validation.
- **DevOps engineers**, to know which deployment, observability, secrets, CDN, and hosting tools are part of the approved stack.
- **Tech leads and architects**, to review whether a proposed tool fits the approved architecture, cost model, reliability needs, and scaling plan.
- **Product managers and founders**, to understand which tools are required for MVP, which are optional, and which may create future cost commitments.

This document should be updated whenever:

- a new tool, API, model, library, or service is introduced
- an existing tool changes from optional to approved
- a free tool becomes paid in practice because of scale or team usage
- a tool is removed, replaced, or rejected
- a new external integration is added, such as a CRM, email provider, meeting platform, or analytics destination
- a local or offline AI option is added for experimentation, fallback, privacy, or cost control
- a major architecture or infrastructure decision changes the stack, such as moving from Railway to AWS or from one LLM provider to another

This document is not meant to explain the full internal implementation of every feature. Its purpose is to answer simple operational questions clearly:

- What are we using
- Why are we using it
- Where is it used
- Is it free, trial-based, offline, online, or paid
- Is it approved for development only, internal demos, testing, or production

New team reading guide (start here)

If you are new to the project, follow this reading order first:

- **All roles (mandatory):** [Section 2](#), [Section 4](#), [Section 5](#), [Section 6](#), then [Section 7](#).
- **Frontend engineers:** [Section 10](#), [Section 14.1-14.4](#), and [Section 17.1-17.3](#).
- **Backend engineers:** [Section 11](#), [Section 13](#), [Section 14.2-14.5](#), [Section 16](#), and [Section 17.1-17.3](#).

- **AI/ML engineers:** [Section 8](#), [Section 9](#), [Section 12](#), [Section 17.6](#), and [Section 20](#).
- **QA engineers:** [Section 10.6](#), [Section 11.5](#), and all of [Section 17](#).
- **DevOps/Security engineers:** [Section 13](#), [Section 14](#), [Section 15](#), [Section 19](#), and [Section 20](#).
- **Tech Leads/Architects:** Full document with priority on [Section 6](#), [Section 7](#), [Section 18](#), and [Section 19](#).

Practical onboarding flow:

- First read core principles, then scan the master table in [Section 7](#), then go deep into your role-specific sections.

3. How to Read This Document

This document lists many kinds of technology. To avoid confusion, use the following definitions.

What is a tool

A **tool** is any software product we directly use during development, testing, deployment, debugging, design, research, or operations. A tool may be local, cloud-based, free, paid, offline, or online.

Examples:

- **Docker** is a tool we use to run local development environments and containerize services for deployment.
- **Playwright** is a tool we use to automate browser-based end-to-end testing.
- **Sentry** is a tool we use to track production errors and performance issues.
- **LM Studio** or **Ollama** are tools we can use to run local AI models during experimentation or fallback testing. [web:11][web:19]

What is a library

A **library** is code that developers import into the application to build product features. Libraries usually live inside the codebase as package dependencies and are not used as standalone products by end users.

Examples:

- **React** is a library used to build frontend user interfaces.
- **Zod** is a library used to validate request and response data.
- **BullMQ** is a library used to manage background jobs and event-driven workflows.
- **LangGraph** is a library used to orchestrate multi-step AI workflows.
- **Whisper**, **spaCy**, and **scikit-learn** are libraries used for speech, NLP, and ML-related development.

What is a managed service

A **managed service** is infrastructure or software hosted and operated by another provider, so we do not need to fully run and maintain it ourselves. We use managed services when they save engineering time, improve reliability, or help us move faster during MVP and early production.

Examples:

- **Supabase** is a managed service for PostgreSQL, Auth, and Storage in our early architecture.
- **Railway** is a managed hosting platform used for Phase 1 and Phase 2 deployment.
- **Upstash Redis** is a managed Redis option for cache and queue backing.
- **Sentry**, **Better Stack**, and **Cloudflare** are managed services for monitoring, logging, uptime, CDN, and edge protection.

Why we use managed services in development:

- faster setup for a small team
- less operational burden
- easier staging and demo environments
- fewer blockers for freshers and interns
- simpler path to MVP delivery

Why we may move away from some managed services later:

- cost at scale
- deeper custom control needs
- performance tuning
- enterprise compliance or deployment constraints

What is an external integration

An **external integration** is a third-party platform, API, or SaaS product that our application connects with to read data, write data, trigger workflows, or sync records. These systems are not part of our core product codebase, but they are essential to platform functionality.

Examples:

- **Salesforce**, **HubSpot**, and **Microsoft Dynamics 365** are CRM integrations.
- **Zoom**, **Microsoft Teams**, and **Google Meet** are meeting and recording integrations.
- **Gmail** and **Outlook / Office 365** are email integrations.
- **Snowflake** and **BigQuery** are data export / warehouse integrations.
- **OpenAI API**, **AssemblyAI**, and evaluation-only provider APIs such as **xAI / Grok routes** are model or AI service integrations. [web:8]

Why we track external integrations separately:

- they often require credentials, OAuth, API quotas, and tenant-specific setup
- they may introduce privacy, compliance, and rate-limit concerns
- they may be customer-owned, partner-owned, or billed separately
- they can break independently of our own code
- they often require special testing and fallback handling

Difference between open source, offline, free online, free trial, and paid

We use the following cost and usage labels throughout this document.

Open source

Open source means the code is publicly available and we can use, inspect, and usually self-host it under its license terms. Open source does **not** always mean zero total cost, because infrastructure, GPUs, storage, and maintenance may still cost money.

Examples:

- Next.js
- NestJS
- FastAPI
- PostgreSQL
- Redis
- BullMQ
- Whisper
- Ollama
- LangGraph [web:19]

Why we use open source:

- lower vendor lock-in
- easier local development
- better learning for freshers
- more flexibility in architecture
- can be self-hosted later if needed

Offline

Offline means the tool or model can run locally on a laptop, workstation, or internal server without requiring a live external API call for every request. Offline tools are especially useful for privacy-sensitive testing, low-cost experimentation, fallback systems, and demos in low-connectivity environments.

Examples:

- Ollama running local models
- LM Studio serving local models on a laptop
- Whisper self-hosted transcription
- local Gemma-family models when hardware supports them [web:11][web:19][web:15]

Why we use offline tools in development:

- reduce API cost during experimentation
- avoid sending sensitive sample data to third parties
- enable local prototyping
- create backup paths when cloud APIs fail
- help engineers understand prompts and model behavior before production rollout

Free online

Free online means a hosted service is available over the internet and has a free usage option, free quota, or no-cost access tier. These tools are useful for evaluation, prototyping, benchmarking, or initial feature experiments, but they often come with quota limits, weaker SLAs, usage restrictions, or data policy concerns.

Examples:

- free hosted AI playgrounds
- free API tiers
- free cloud inference options
- free hosted dashboards or developer plans [web:8][web:17]

Why we use free online tools:

- fast proof-of-concept work
- compare multiple APIs before paying
- test model quality quickly
- avoid self-hosting too early

Why we use them carefully:

- free limits can disappear or change
- latency and reliability may not be stable
- terms of service may restrict production use
- sensitive customer data should not be sent without approval

Free trial / free tier

Free trial means the provider gives temporary access for evaluation. Free tier means the provider allows ongoing limited usage before paid upgrade is required. These tools are useful for MVP and early testing, but we should assume they may become paid as soon as usage grows.

Examples:

- Supabase free tier
- Railway starter usage
- Sentry free tier
- Better Stack starter plans
- Vercel starter usage
- managed AI API credits or introductory usage

Why we use free tier / trial services:

- quick setup
- low initial cash burn
- easy team onboarding
- enough for staging, QA, and prototype demos

Why we track them carefully:

- they can become hidden future costs
- team usage often exceeds free limits faster than expected
- some features required for production are locked behind paid plans

Paid

Paid means the tool or service requires direct subscription fees, usage-based billing, enterprise licensing, or infrastructure spend. Some paid tools are worth it from day one because they save major development time or provide core platform functionality. Others should only be adopted after clear need.

Examples:

- OpenAI API for production LLM usage
- managed cloud hosting beyond free limits
- enterprise observability tools
- premium CRM or sales ecosystem integrations
- GPU infrastructure for heavy inference workloads

Why we choose paid tools:

- better reliability and support
- stronger SLAs
- production-grade scale
- faster time to market
- less engineering maintenance overhead

How to interpret each tool entry

Each tool entry in this document should answer these simple questions:

- **What is it**
- **Where do we use it**
- **Why do we use it**
- **Is it for development, testing, internal demos, or production**
- **Is it offline, open source, free online, free trial, or paid**
- **What problem does it solve for our team**
- **What is the recommended use case**

Example of how a tool should be documented later in this file:

- **Tool:** Ollama
- **Type:** Free offline AI runtime

- **Used in:** Local AI experimentation, fallback testing, private demos
- **Why we use it:** It lets us run open models locally through a simple API, which helps reduce API cost and makes local testing easier.
- **Typical use cases:** prompt testing, local summarization experiments, backup LLM path, private internal demos
- **Usage note:** Good for development and evaluation; not automatically approved for production without review. [web:19]
- **Tool:** OpenAI API
- **Type:** Paid managed AI API
- **Used in:** Production-grade LLM features such as summaries, RAG answers, structured extraction, and AI agents
- **Why we use it:** It provides the primary cloud LLM capability defined in the architecture and offers strong model quality for user-facing AI features.
- **Typical use cases:** call summaries, AI email drafts, query answering, extraction, agent workflows
- **Usage note:** Paid by usage and must be monitored for token cost.

4. Cost Classification

This section groups tools and services by cost model so the team can quickly decide what we can start using immediately, what can be used only for experimentation, what is safe for MVP, and what will create future cost commitments. This is important because the platform architecture already combines open-source tools, managed services, external APIs, and future scale-up infrastructure, so cost control must be visible from the beginning.

A tool can appear in more than one mindset but should have one primary cost label in this document. For example, an open-source tool may still require paid infrastructure, and a free-tier service may later become a paid production dependency. The goal of this section is practical classification, not perfect accounting.

4.1 Free and Open Source

Free and open-source tools are the foundation of our engineering stack because they reduce early cost, improve developer learning, and give us flexibility to self-host or customize when needed. They are especially useful for local development, internal testing, and building the first version of the platform without locking ourselves too early into expensive vendors.

We use free and open-source tools mainly when:

- we want fast local setup
- we want developers and freshers to learn the stack deeply
- we want strong control over architecture
- we want to reduce vendor lock-in
- we want low-cost experimentation before committing to managed services

Typical examples in our stack include:

- **Next.js** for building the frontend application because it gives us a modern React-based product foundation with server-side rendering and routing.
- **React** because it is the core UI library for building reusable product interfaces.
- **Tailwind CSS** because it helps the team build consistent UI quickly without writing large custom CSS files.
- **NestJS** because it gives us a strong modular backend structure aligned with our architecture.
- **FastAPI** because it is a clean and productive framework for AI and ML services.

- **PostgreSQL** because it is a reliable relational database and supports extensions like pgvector.
- **Redis** because it supports caching and queues.
- **BullMQ** because we use event-driven and background processing heavily.
- **Prisma** because it gives type-safe database access and migration workflows.
- **Zod** because it helps validate data contracts clearly.
- **Playwright, Jest, pytest, and k6** because testing must be part of the platform from the start.
- **Whisper** because it gives us open transcription capability.
- **LangGraph** and **LiteLLM** because they help structure AI workflows and provider routing in code.
- **Ollama** because it allows us to run open models locally for development and fallback experiments.

[web:26]

Important note: open source does not always mean zero cost. We may still pay for hosting, storage, GPU machines, or engineering effort to run these tools at scale.

4.2 Free Offline AI Tools

Free offline AI tools are tools and model runtimes that can run locally on a laptop, workstation, or internal server without requiring a cloud API for every request. These are useful for local experimentation, private demos, prompt development, cost control, and fallback paths when external model APIs are unavailable. Our architecture already anticipates local or self-hosted fallback patterns, especially through the planned Ollama-based model path.

We use free offline AI tools mainly when:

- we want to reduce API spend during development
- we want to test prompts privately
- we want a local backup if cloud AI providers fail
- we want engineers to experiment quickly without waiting for API approvals
- we want to avoid sending sample customer-like data to external services during early prototyping

Typical examples:

- **Ollama** because it provides a simple local runtime and API for open models, making it great for prompt testing, local summarization experiments, offline demos, and fallback inference. [web:26]
- **LM Studio** because it gives a desktop-friendly way to run and test local models, which is useful for non-backend engineers and product stakeholders during experimentation. [web:11]
- **Gemma-family models** because they can be used for local evaluation, summarization experiments, lightweight assistant workflows, and private testing when hardware is sufficient. [web:5][web:15]
- **Whisper self-hosted** because we need offline or self-controlled transcription for testing and for reducing dependence on third-party speech APIs.
- **pyannote.audio** because it helps with speaker diarization in local or controlled AI pipelines.
- **llama.cpp** style runtimes or similar local inference tools because they make CPU-based or lightweight local experiments possible when GPU capacity is limited. [web:17][web:29]

Recommended offline use cases:

- local prompt engineering
- internal POC demos
- lightweight AI feature validation before cloud rollout
- privacy-sensitive transcript experiments
- backup LLM path in development
- model comparison without immediate API cost

Important note: offline AI is excellent for development and fallback strategy, but it is not automatically the best choice for production. Local models may have weaker quality, slower inference, hardware limitations, or more operational work than cloud APIs.

4.3 Free Online AI Tools / APIs

Free online AI tools and APIs are hosted services that offer free access, developer quotas, or no-cost experimentation over the internet. These are extremely useful for evaluation, benchmarking, comparison testing, and quick proof-of-concept work. However, they should be treated carefully because free quotas, latency, terms of service, and privacy guarantees can change. [web:20][web:23]

We use free online AI tools mainly when:

- we want to quickly compare models before paying
- we want to prototype AI features fast
- we want to benchmark speed, accuracy, or context-window behavior
- we want easy access to models without self-hosting
- we want free inference for non-sensitive experiments

Typical examples:

- **Google AI Studio** because it offers generous free access for prototyping, prompt testing, multimodal experiments, and model evaluation without requiring production-grade setup at the start. [web:20][web:22][web:23]
- **Groq** because it provides fast inference for open models and is useful when we want to benchmark latency-sensitive workflows such as quick extraction or low-latency chat patterns. [web:20][web:21]
- **Hugging Face Inference** because it gives access to a wide ecosystem of hosted open-source models, which is useful for testing specialized tasks like summarization, translation, sentiment analysis, or named entity extraction. [web:20][web:24]
- **OpenRouter** because it can help compare multiple models through a unified API and can be useful for evaluation and fallback experiments, though it should not be assumed to be our primary production provider without review. [web:27][web:30]
- **Free hosted model hubs and playgrounds** because they help the team quickly validate whether a model family is promising before deeper integration.

Recommended free online use cases:

- model benchmarking
- prompt experiments
- temporary prototypes
- comparing open models vs paid models
- validating task quality before architecture decisions
- testing speed and context handling

Important caution:

- Free online AI tools are useful for experimentation, not automatic production approval.
- They should not receive sensitive customer data unless specifically approved.
- We must document quotas, data policies, reliability limits, and migration risks before depending on them.

4.4 Free Tier / Free Trial

Free tier and free trial services are hosted tools that help us move quickly during MVP, early testing, and internal deployments while keeping initial cost low. These services are useful because they reduce DevOps burden and let small teams ship faster, but we must plan for the point where usage, seats, or production features force an upgrade.

We use free tier or trial tools mainly when:

- we need fast setup
- we want low upfront spend
- we need staging or demo environments
- we do not yet have enough scale to justify full paid plans
- we want to evaluate a service before standardizing on it

Typical examples:

- **Supabase** because it offers a fast path for managed PostgreSQL, authentication, and storage in early phases.
- **Railway** because it is easy to use for early service deployment and internal environments before larger-scale infrastructure is required.
- **Vercel** because it is convenient for frontend deployment and preview environments.
- **Sentry** because it provides quick error monitoring with low initial setup.
- **Better Stack** because it can provide early uptime monitoring and logging workflows before full enterprise observability maturity.
- **Cloudflare** because even basic plans can provide useful CDN, TLS, and edge protections.
- **Resend** or similar email delivery tools because they often support small-scale transactional email during MVP.
- **AI API free credits** because they help compare providers before usage becomes meaningful.

Recommended free tier / trial use cases:

- MVP hosting
- staging environments
- preview deployments
- initial monitoring
- internal QA
- startup demos
- lightweight transactional workflows

Important note:

- Every free tier service should have an expected upgrade trigger documented later in this file.
- If a free tier tool becomes critical to production, we should assume it will eventually become a paid item.

4.5 Paid Now

Paid now tools are tools or services we either already know we need for core product functionality or should assume will create real cost during development and production. These are worth paying for because they unlock critical product features, save major engineering effort, or provide quality and reliability we cannot easily replace with free alternatives.

We use paid-now tools mainly when:

- the product depends directly on them
- their quality is important to user experience
- the team would lose too much time replacing them
- they provide strong reliability, speed, or support
- they are central to AI or production operations

Typical examples:

- **OpenAI API** because the architecture defines it as the primary LLM provider for major AI features such as summaries, question answering, extraction, and agentic workflows.
- **AssemblyAI fallback usage** because it is part of the fallback speech pipeline when Whisper is unavailable or insufficient.
- **Cloud hosting beyond hobby limits** when the app reaches real internal or customer traffic.
- **Production observability plans** when free logging or alerting limits are not enough.
- **GPU infrastructure** when self-hosted transcription or local model serving becomes a real workload.
- **xAI / Grok API** (evaluation only; production use requires Tech Lead approval) because public pricing indicates token-based billing and promotional credits are not a permanent plan. [web:25][web:28][web:31]

Recommended paid-now use cases:

- production LLM features
- production fallback AI services
- serious monitoring
- real user-facing staging and production environments
- high-value integrations that save development time or improve quality

Important note:

- Paid-now tools should have cost monitoring, owner assignment, and clear business justification.
- Any usage-based AI provider should have token and request tracking from the beginning.

4.6 Paid Later at Scale

Paid later at scale tools are tools we may not need on day one, but we should expect to adopt when tenant count, traffic volume, uptime requirements, analytics volume, or security expectations increase. Our architecture already anticipates several of these future transitions, including Railway to AWS ECS and possible vector-store upgrades beyond pgvector when scale demands it.

We use paid-later tools when:

- the MVP outgrows starter hosting
- production reliability requirements increase
- team size increases
- customer SLAs become stricter
- observability and incident handling need stronger workflows
- performance bottlenecks justify more specialized systems

Typical examples:

- **AWS ECS / Fargate** because the architecture plans a move there in later phases when load and tenant counts justify more scalable infrastructure.
- **Upstash HA / advanced Redis setup** because higher reliability may be needed for queues and caching.
- **Cloudflare Pro or higher** because stronger WAF, edge controls, and production-grade protections may become necessary.
- **Doppler Team or advanced secrets management** because environment and team complexity increases over time.
- **PagerDuty or stronger incident tooling** because production support workflows mature with scale.
- **Managed ClickHouse or larger analytics infrastructure** because dashboard and reporting workloads grow.
- **Alternative vector stores such as Qdrant, Weaviate, or Pinecone** if pgvector limits are reached for larger enterprise workloads.
- **Dedicated GPU inference clusters** if local or self-hosted AI workloads become large enough to justify them.

Recommended paid-later use cases:

- scale-out production hosting
- enterprise observability
- high-volume search or analytics
- advanced multi-tenant performance isolation
- stronger SRE and incident response
- large embedding or RAG workloads

Important note:

- Paid-later does not mean optional forever.
- It means we should not spend there too early, but we should design so that migration is straightforward when needed.

5. Usage Classification

This section groups tools by functional role in the project. A tool may belong to more than one category, but every tool should have one primary usage label to keep the document easy to maintain. This helps the team understand not just what a tool costs, but also why it exists in the stack and which team depends on it most.

5.1 Development Tools

Development tools are the tools we use to build product features, run the application locally, write code, validate contracts, and support day-to-day engineering work. These tools are the primary foundation for implementation speed and developer productivity.

We use development tools mainly for:

- local coding
- running services locally
- validating schemas and contracts
- building frontend and backend features
- debugging during feature implementation
- onboarding new engineers quickly

Typical development tools and why we use them:

- **Node.js** because it runs our TypeScript product services and frontend tooling stack.
- **TypeScript** because it gives type safety across frontend and backend development, reducing bugs during implementation.
- **Python** because it is the main language for AI and ML service development in the architecture.
- **Next.js** because it provides the framework for our frontend application.
- **NestJS** because it supports modular backend architecture and clean service boundaries.
- **FastAPI** because it is productive for internal AI service endpoints.
- **Prisma** because it helps us work with PostgreSQL using typed models and migrations.
- **Zod** because it validates request, response, and configuration shapes clearly.
- **Docker Compose** because it helps developers run the stack locally with fewer setup issues.
- **Ollama** because it allows developers to experiment with local AI models without always calling paid APIs. [web:26]

5.2 Testing Tools

Testing tools are used to verify that the system works correctly at unit, integration, end-to-end, and performance levels. Because the platform is modular and integration-heavy, testing tools are essential, not optional.

We use testing tools mainly for:

- unit testing business logic
- integration testing APIs and services
- end-to-end workflow validation
- browser automation
- performance and load testing
- regression prevention

Typical testing tools and why we use them:

- **Jest** because it is a standard and productive framework for backend and shared TypeScript testing.
- **Supertest** because it helps test API endpoints and request/response behavior.
- **pytest** because it is the standard for Python-based AI and ML service testing.
- **pytest-asyncio** because many AI service workflows are async.
- **Playwright** because it is strong for realistic end-to-end browser automation.

- **k6** because it helps measure load, throughput, and performance behavior.
- **Testcontainers** because it allows realistic integration tests using real services like PostgreSQL or Redis in containers.
- **Mocking and fixture tools** because external integrations and AI calls often need deterministic test behavior.

5.3 AI / ML Tools

AI / ML tools are used to build, run, test, and improve the platform's AI features. These include model APIs, local model runtimes, agent frameworks, speech tools, embedding tools, and classical ML libraries. This category is especially important because AI is central to summaries, trackers, extraction, RAG, forecasting, and training workflows in the product.

We use AI / ML tools mainly for:

- call summarization
- query answering
- AI email generation
- tracker detection
- topic and theme extraction
- forecasting support
- embeddings and retrieval
- speech-to-text and diarization
- evaluation and fallback experiments

Typical AI / ML tools and why we use them:

- **OpenAI API** because it is the primary cloud LLM provider defined in the architecture.
- **LiteLLM** because it gives a unified interface for calling multiple providers and managing fallback behavior in code.
- **LangGraph** because it supports structured multi-step AI workflows and agents.
- **Whisper** because it provides transcription capability.
- **AssemblyAI** because it serves as a fallback transcription route when needed.
- **spaCy** because it helps with NLP preprocessing and extraction workflows.
- **sentence-transformers** and embedding tools because semantic retrieval and RAG need embeddings.
- **pgvector** because it lets us store embeddings close to source data in PostgreSQL.
- **Ollama**, **LM Studio**, and local open models because they are useful for experimentation, fallback design, and offline development. [web:26][web:11]
- **scikit-learn** and **XGBoost** because forecasting and predictive tasks are better handled by classical ML in some cases than by LLMs.

5.4 Infrastructure / DevOps Tools

Infrastructure and DevOps tools are used to package, deploy, run, and manage the application across local, staging, and production environments. They are critical because even a good codebase fails without reliable deployment and operations discipline.

We use infrastructure / DevOps tools mainly for:

- containerization
- deployment automation
- environment management
- scaling services
- release workflows
- infrastructure consistency
- secrets and runtime configuration

Typical infrastructure / DevOps tools and why we use them:

- **Docker** because all major services are containerized.
- **Docker Compose** because it simplifies local orchestration.
- **Railway** because it supports fast early deployment in Phase 1 and Phase 2.
- **Vercel** because it supports frontend deployment and preview workflows.
- **GitHub Actions** because it automates CI/CD checks such as linting, tests, validation, and deploys.
- **Doppler** because secrets management should be centralized and not hardcoded into repos or images.
- **AWS ECS / Fargate** because later scale may require more robust hosting and control.
- **Cloudflare** because it can provide edge, CDN, TLS, and protection capabilities.

5.5 Data and Storage Tools

Data and storage tools handle the system of record, caching, indexing, analytics, embeddings, files, and exports. Because the product is built around capturing and structuring revenue signals, this category is central to the architecture.

We use data and storage tools mainly for:

- transactional data storage
- multi-tenant isolation
- vector storage
- search indexing
- analytics queries
- temporary caching and queues
- file and recording storage
- warehouse export workflows

Typical data and storage tools and why we use them:

- **PostgreSQL / Supabase PostgreSQL** because it is the primary relational store for platform data.
- **pgvector** because semantic search and RAG need vector storage near core data.
- **Redis** because it supports caching, rate limits, and queue backing.
- **BullMQ** because background jobs need durable queue support.
- **Meilisearch** because full-text search is important for conversation library and filtered discovery.
- **ClickHouse** because analytical workloads benefit from a columnar engine.
- **Supabase Storage** or equivalent object storage because recordings and large files should not live directly in the application database.
- **Warehouse connectors** because customers may need exports to systems like Snowflake or BigQuery.

5.6 Monitoring and Security Tools

Monitoring and security tools help us understand system health, detect incidents, protect environments, and enforce safe operations. In a multi-tenant AI platform, these tools are essential for trust and reliability.

We use monitoring and security tools mainly for:

- error tracking
- uptime monitoring
- log collection
- alerting
- rate limiting
- secrets protection
- incident response
- CDN and WAF protection

Typical monitoring and security tools and why we use them:

- **Sentry** because it gives visibility into production errors and performance problems.

- **Better Stack** because it supports log aggregation, uptime monitoring, alerting, and operational visibility.
- **Grafana** because metrics dashboards help us understand queue depth, latency, and service health.
- **Cloudflare** because it provides TLS termination, CDN features, DDoS protection, and edge controls.
- **Doppler** because secrets must be managed centrally and securely.
- **PagerDuty** or equivalent later because incident response maturity increases as production scale grows.

5.7 External APIs and Integrations

External APIs and integrations connect the platform to systems we do not own but need for core workflows. This is a major category because the product depends heavily on CRM data, meeting systems, email systems, and external AI services.

We use external APIs and integrations mainly for:

- CRM sync
- call recording ingestion
- meeting metadata capture
- email sending and syncing
- external AI model access
- warehouse export
- collaboration and notification workflows

Typical external APIs and integrations and why we use them:

- **Salesforce, HubSpot, and Dynamics 365** because CRM sync is core to revenue workflows.
- **Zoom, Microsoft Teams, and Google Meet** because the platform captures meeting and call data from these ecosystems.
- **Gmail API and Outlook / Microsoft Graph** because email workflows are part of capture and engagement features.
- **OpenAI API** because it powers core AI tasks in the initial architecture.
- **AssemblyAI** because it supports fallback speech workflows.
- **Snowflake and BigQuery** connectors because customers may need downstream reporting or warehouse exports.
- **Slack** or similar notification tools because alerts and workflow outputs may need collaboration integration.
- **xAI / Grok routes, Google AI Studio**, or similar AI providers may be used for evaluation only and must be reviewed before any production dependency. [web:22][web:25]

5.8 Productivity / Team Tools

Productivity and team tools help engineers, QA, PMs, and leads work together effectively, even though they are not always part of the runtime product stack. They still matter because poor collaboration creates delivery problems, onboarding friction, and knowledge gaps.

We use productivity and team tools mainly for:

- documentation
- issue tracking
- collaboration
- design handoff
- architecture review
- meeting coordination
- code review workflow

Typical productivity and team tools and why we use them:

- **GitHub** because it is the center of source control, pull requests, and engineering collaboration.

- **GitHub Projects / Jira** because tasks and sprint execution need visibility.
- **Notion / Confluence / Markdown docs** because architecture and decisions must be documented.
- **Figma** because UI and UX handoff is important for frontend development.
- **Slack / Teams** because engineering communication and operational coordination require a shared channel.
- **Google Sheets / Excel** because procurement, planning, and tool tracking are often easiest to manage in structured tables before being formalized in code or docs.

Important note:

- Productivity tools should also be tracked here if they create recurring seat cost, workflow dependency, or documentation lock-in.
- Just because a tool is “not part of runtime” does not mean it is unimportant.

6. Core Principles

This document is not just a list of tools. It is also a decision guide. These core principles help the team choose tools in a disciplined way so we do not end up with unnecessary complexity, duplicated services, hidden costs, or unsupported technology choices. The architecture already defines a standard stack for the platform and states that every technology decision should be governed and reviewed before it enters the codebase, so this inventory should follow the same discipline.

Prefer approved stack first

Always check the approved platform stack before proposing a new tool, library, model, API, or service. The current architecture already defines standard choices across frontend, backend, AI services, storage, search, hosting, observability, and secrets management, including tools such as Next.js, NestJS, FastAPI, PostgreSQL, BullMQ, LiteLLM, Whisper, Meilisearch, ClickHouse, Railway, Sentry, Better Stack, and Doppler.

Why this principle exists:

- it reduces unnecessary tool sprawl
- it keeps onboarding easier for freshers and new team members
- it improves maintainability because more engineers understand the same stack
- it reduces integration and operational risk
- it keeps architecture decisions consistent across modules

Practical rule:

- If the tool already exists in the approved stack, use it unless there is a strong reason not to.
- If two tools solve the same problem, prefer the one already approved and already used in the platform.
- Do not add a second tool just because it looks newer, trendier, or easier in one small case.

Do not introduce new tools without review

No new tool should be added casually to the codebase, infrastructure, or workflow. The architecture document explicitly states that every technology must have an approved architecture decision before it enters the codebase, and that new alternatives should not be introduced without review.

Why this principle exists:

- every new tool creates maintenance cost
- every new vendor creates cost and legal review risk
- every new API creates security and reliability considerations
- every new library increases upgrade and compatibility burden
- every new AI provider increases prompt, evaluation, and fallback complexity

Practical rule:

- New tools must be reviewed by the Tech Lead or the designated architecture owner.
- The proposal should clearly explain:
 - what problem the tool solves
 - why the current approved stack is not enough
 - cost impact
 - security and privacy impact
 - migration or lock-in risk
 - whether it is for development only, testing, internal use, or production
- If the tool touches AI, data, auth, observability, infra, or external integrations, review is mandatory before adoption.

Version pinning and version governance are mandatory

The stack spans JavaScript, Python, containers, and managed services. Without explicit version governance, environment drift creates avoidable CI failures and "works on my machine" bugs.

Practical rule:

- Every approved dependency must be explicitly versioned in the appropriate manifest or config (`package.json`, lock files, `pyproject.toml`, `requirements*.txt`, Docker image tags, and `docker-compose.yml`).
- Do not use floating `latest` tags for production dependencies.
- Runtime versions (Node, Python, PostgreSQL, Redis) must be documented and aligned between local development and CI.
- Major version changes require Tech Lead review before merge.

Upgrade governance and dependency ownership are mandatory

High-impact tools such as NestJS, FastAPI, LiteLLM, LangGraph, and provider SDKs need controlled upgrade ownership.

Practical rule:

- Assign a clear owner for each production dependency (Frontend Lead, Backend Lead, AI Lead, DevOps Lead, or Security Owner).
- Minor/patch upgrades can be batched in maintenance windows.
- Major upgrades require:
 - impact note (breaking changes)
 - migration plan
 - rollback plan
 - test plan and validation evidence
 - Tech Lead approval before production rollout
- Security fixes take priority over feature upgrades.
- CI must run dependency vulnerability checks using `npm audit` (or equivalent JS tooling) and `pip-audit` for Python services.

Prefer open-source for local development

For local development, prefer open-source tools whenever practical. The architecture already leans heavily on open-source foundations such as Next.js, NestJS, FastAPI, PostgreSQL, Redis, BullMQ, Prisma, Zod, Whisper, LangGraph, and other code-level dependencies, which makes this a natural principle for the team.

Why this principle exists:

- it lowers early cost

- it makes local setup easier
- it helps new engineers learn the actual system instead of depending only on hosted black-box tools
- it reduces vendor dependence during MVP
- it gives us freedom to self-host or customize later

Good examples:

- use **PostgreSQL** locally instead of depending only on a remote DB
- use **Redis** and **BullMQ** locally for event and queue testing
- use **Docker Compose** to run the stack locally
- use **Whisper** locally for transcription experiments where possible
- use **Ollama** for local open-model testing instead of always calling paid LLM APIs [web:26]

Practical rule:

- If a tool is only needed for local development and a stable open-source option exists, prefer that option first.
- Use hosted-only developer tools when they save significant time or solve a problem that local tools cannot solve reasonably.

Prefer free offline AI for experimentation, privacy, and backup workflows

Whenever possible, use free offline AI tools for early experimentation, prompt testing, internal demos, and backup workflows. This is especially useful because the platform depends heavily on AI, and cloud model costs can rise quickly during testing. The architecture already supports self-hosted and fallback-oriented thinking, including local transcription and planned model fallback strategies.

Why this principle exists:

- it reduces token and API costs during experimentation
- it helps protect privacy for sample or sensitive internal data
- it gives the team a fallback path when external AI APIs fail
- it allows faster prompt iteration without waiting for API approvals or billing setup
- it helps compare open models against paid providers before committing

Good examples:

- use **Ollama** to run local models for summarization tests, prompt debugging, and private experiments [web:26]
- use **LM Studio** when product or non-backend team members need an easier local testing experience [web:11]
- use **Gemma-family models** or similar open models for controlled experimentation if local hardware supports them [web:5][web:15]
- use **Whisper** for self-hosted transcription experiments and fallback speech workflows

Practical rule:

- Offline AI is preferred for:
 - internal prototyping
 - evaluation
 - backup design
 - privacy-sensitive testing
 - prompt experiments
- Offline AI is not automatically approved for production.
- Before production adoption, evaluate quality, latency, hardware cost, scaling effort, and operational support needs.

Prefer free online AI only for low-risk prototyping

Free online AI tools and APIs are useful for quick experimentation, but they should be treated as low-risk prototyping tools unless they go through explicit review. Free hosted AI services can change quotas, latency, availability, and terms without warning, so they are best used for evaluation rather than core dependency by default. [web:20][web:23]

Why this principle exists:

- free quotas can disappear
- reliability is often weaker than paid plans
- privacy and retention policies may not suit customer data
- API behavior can change without enterprise guarantees
- prototyping convenience can create accidental production dependency

Good examples:

- use **Google AI Studio** for fast prompt exploration and model testing [web:22]
- use **Hugging Face hosted inference** to compare open models quickly [web:24]
- use **Groq** or similar free developer-friendly inference services to benchmark speed [web:20][web:21]
- use optional providers such as **xAI / Grok routes** only for evaluation unless specifically approved for production use [web:25][web:31]

Practical rule:

- Free online AI tools are acceptable for:
 - early model comparison
 - internal experiments
 - non-sensitive prototypes
 - hackathon-style validation
- Do not send sensitive customer or tenant data to free online AI services without approval.
- Do not build production-critical workflows on free online AI quotas alone.

Move to paid tools only when justified by scale, security, reliability, or product quality

Paid tools are not bad; they just need clear justification. The architecture already assumes that some services will be paid because they provide core product value or become necessary at scale, such as OpenAI API, managed hosting, production observability, secrets management, advanced CDN/WAF features, and eventually larger infrastructure such as AWS ECS.

Why this principle exists:

- we should preserve cost efficiency during MVP
- we should not over-engineer before the product proves demand
- we should pay where it meaningfully improves delivery speed or customer experience
- we should avoid hidden recurring costs from tools we do not truly need

Good reasons to move to paid tools:

- the free tier is no longer enough for production load
- the product needs better reliability or SLA support
- the team needs stronger security or compliance capabilities
- a paid model or service clearly delivers better output quality
- self-hosting would cost more engineering time than the paid option saves
- customer-facing latency, uptime, or monitoring expectations increase

Examples:

- use **OpenAI API** for production-grade summarization and structured outputs where model quality matters most

- upgrade **Sentry**, **Better Stack**, or **Cloudflare** when production reliability and visibility requirements increase
- move from **Railway** to **AWS ECS / Fargate** when scale or operational requirements justify it
- upgrade vector or analytics infrastructure when data size and performance needs exceed the MVP architecture limits

Practical rule:

- Before adopting a paid tool, document:
 - business reason
 - technical reason
 - expected monthly or usage cost
 - owner
 - upgrade trigger
 - rollback or replacement option if possible

7. Inventory Summary Table

This table is the quick-glance index of the entire document. Every major tool, library, service, model, platform, and integration should eventually appear here with a short purpose and a clear owner. The goal is to make it easy for anyone to answer simple questions such as “What do we use for this?”, “Why do we need it?”, and “Is this free or paid?” without reading the whole document. It also connects the cost labels from Section 4 and the usage labels from Section 5 directly to each tool row.

Use the table with these rules:

- One row per tool or service.
- Keep the **Primary Use** field short and simple.
- Use **Key Use Cases** to explain where the tool helps us in development, testing, AI, deployment, or production.
- Use **Owner** to indicate the primary responsible team, such as Frontend, Backend, AI, DevOps, QA, Security, or Product.
- Use **Security Sensitivity** to flag handling risk (**Low**, **Medium**, **High - handles customer data**, **High - holds credentials**).
- Use **Notes** for approval status, restrictions, migration plans, or warnings.
- If a tool is only for experimentation, say so clearly.
- If a tool should not receive customer data, mention that in Notes.

Table field meanings

- **Category**: Broad functional bucket such as Frontend, Backend, AI/ML, Data, Infra, Monitoring, Integration, or Productivity.
- **Tool / Service**: Name of the tool, library, platform, API, model provider, or managed service.
- **Type**: What kind of thing it is, such as framework, library, local runtime, managed service, external API, or hosted platform.
- **Offline**: Use **Yes**, **No**, or **Partial**. Use **Yes** if it can work fully locally for its main use case.
- **Open Source**: Use **Yes**, **No**, or **Partial**.
- **Free Tier / Trial**: Use **Yes**, **No**, or **N/A**.
- **Paid Now**: Use **Yes** if we already expect real cost in active development or production.
- **Paid Later**: Use **Yes** if we expect cost mainly after scale, enterprise rollout, or infrastructure growth.
- **Primary Use**: The main reason we keep this tool in the stack.
- **Key Use Cases**: Practical examples of how we use the tool in development, testing, AI workflows, deployment, or operations.
- **Owner**: Team primarily responsible for the tool.
- **Security Sensitivity**: Use **Low**, **Medium**, **High - handles customer data**, or **High - holds credentials**.
- **Notes**: Limits, approvals, privacy restrictions, migration triggers, or special comments.

Master inventory table (consolidated from Sections 8 through 17)

Category	Tool / Service	Type	Offline	Open Source	Free Tier / Trial	Paid Now	Paid Later	Primary Use
AI / ML	Ollama	Local runtime	Yes	Partial	N/A	No	Possible	Local open-model runtime
AI / ML	LM Studio	Local desktop runtime	Yes	No	N/A	No	No	Local AI testing UX
AI / ML	llama.cpp-compatible runtimes	Local runtime family	Yes	Yes	N/A	No	No	Lightweight local inference
AI / ML	Gemma 3	Open model family	Yes	Partial	Yes	No	Possible	Starter local model
AI / ML	Gemma 4	Open model family	Yes	Partial	Yes	No	Possible	Advanced local model
AI / ML	Llama / Qwen / Mistral / DeepSeek / Phi	Open model families	Yes	Partial	Yes	No	Possible	Model family comparison
AI / ML	Whisper	ASR model/library	Yes	Yes	N/A	Yes	Yes	Speech-to-text
AI / ML	AssemblyAI	Managed ASR API	No	No	Possible	Yes	Yes	Backup speech provider
AI / ML	pyannote.audio	Diarization library	Yes	Yes	N/A	Possible	Yes	Speaker diarization
AI / ML	FFmpeg	Media processing tool	Yes	Yes	N/A	No	No	Audio preprocessing
AI / ML	Google AI Studio	Hosted AI platform	No	No	Yes	No	Possible	Fast online prototyping
AI / ML	Groq	Hosted inference API	No	No	Yes	No	Possible	Latency benchmarking
AI / ML	Hugging Face hosted inference	Hosted model API	No	No	Yes	No	Possible	Hosted model experiments
AI / ML	xAI / Grok API routes	Hosted AI API	No	No	Possible	No	Possible	Alternative provider evaluation
AI / ML	Hugging Face Hub	Model hub	No	No	Yes	No	Possible	Model discovery
AI / ML	Kaggle model distribution path	Model access platform	No	No	Yes	No	Possible	Access Gemma assets

Category	Tool / Service	Type	Offline	Open Source	Free Tier / Trial	Paid Now	Paid Later	Primary Use
Frontend	Next.js 15	Framework	No	Yes	N/A	No	No	Frontend foundation
Frontend	TypeScript	Language/type system	Yes	Yes	N/A	No	No	Type-safe frontend code
Frontend	Tailwind CSS	Styling framework	Yes	Yes	N/A	No	No	UI styling standard
Frontend	shadcn/ui	Component system	Yes	Yes	N/A	No	No	Reusable UI components
Frontend	Radix UI	Accessibility primitives	Yes	Yes	N/A	No	No	Accessible interactions
Frontend	TanStack Query	Server-state library	Yes	Yes	N/A	No	No	API state management
Frontend	Zustand	Client-state library	Yes	Yes	N/A	No	No	UI state management
Frontend	Next.js router/search params	URL state mechanism	Yes	Partial	N/A	No	No	Shareable state in URL
Frontend	React Hook Form	Form library	Yes	Yes	N/A	No	No	Form state/validation integration
Shared	Zod	Validation library	Yes	Yes	N/A	No	No	Runtime contract validation
Frontend	Recharts	Chart library	Yes	Yes	N/A	No	No	Dashboard visualization
QA	Playwright	E2E test tool	Yes	Yes	N/A	No	Possible	Browser journey testing
QA	Jest	Unit/component test framework	Yes	Yes	N/A	No	No	Unit testing
Backend	NestJS v11	Backend framework	Yes	Yes	N/A	No	No	Modular API services
Backend	Prisma ORM	ORM + migrations	Yes	Partial	N/A	No	Possible	DB access and migrations
Backend	REST API pattern	API design pattern	Yes	Partial	N/A	No	No	Stable service contracts
Backend	Swagger/OpenAPI tooling	API docs tooling	Yes	Partial	N/A	No	Possible	API discoverability
Security	Supabase Auth	Managed auth service	No	No	Yes	Yes	Yes	Identity and sessions

Category	Tool / Service	Type	Offline	Open Source	Free Tier / Trial	Paid Now	Paid Later	Primary Use
Security	JWT Guards	Authorization mechanism	Yes	Partial	N/A	No	No	Route protection
Security	RBAC	Access control model	Yes	Partial	N/A	No	No	Role-scoped permissions
Backend / AI	BullMQ	Queue/event library	Yes	Yes	N/A	No	Possible	Async orchestration
Backend / AI	Redis	Cache and queue store	Yes	Yes	N/A	Yes	Yes	Queue backing + caching
AI / ML	OpenAI API	Managed LLM API	No	No	No	Yes	Yes	Primary cloud LLM
AI / ML	LiteLLM	LLM gateway/routing framework	Yes	Yes	N/A	No	Possible	Provider abstraction
AI / ML	Anthropic fallback path	Secondary LLM provider	No	No	No	Possible	Yes	Resilience fallback
AI / ML	Python 3.12	AI runtime language	Yes	Yes	N/A	No	No	AI service implementation
AI / ML	FastAPI	AI service framework	Yes	Yes	N/A	No	No	Internal AI APIs
AI / ML	LangGraph	Agent/workflow framework	Yes	Yes	N/A	Possible	Yes	Multi-step AI orchestration
AI / ML	spaCy	NLP preprocessing library	Yes	Yes	N/A	No	No	Deterministic NLP
AI / ML	sentence-transformers	Embedding library	Yes	Yes	N/A	No	Possible	Local embeddings

Category	Tool / Service	Type	Offline	Open Source	Free Tier / Trial	Paid Now	Paid Later	Primary Use
Data	pgvector	Vector extension	Yes	Yes	N/A	No	Yes	Embedding retrieval in Postgres
AI / ML	scikit-learn	ML library	Yes	Yes	N/A	No	No	Classical ML models
AI / ML	XGBoost	ML library	Yes	Yes	N/A	No	Possible	Forecasting and scoring
Data	PostgreSQL	Relational database	Yes	Yes	N/A	Yes	Yes	Source-of-truth datastore
Data / Security	Shared PostgreSQL + RLS	Data isolation pattern	Yes	Partial	N/A	No	No	Tenant isolation
Data	Supabase PostgreSQL	Managed DB	No	No	Yes	Yes	Yes	Managed relational DB ops
Infra	Upstash Redis HA	Managed Redis option	No	No	Yes	Possible	Yes	High availability Redis
Search	Meilisearch	Search engine	Yes	Yes	N/A	Possible	Yes	Product search
Analytics	ClickHouse	Analytics store	Yes	Yes	N/A	Possible	Yes	Analytical queries
Storage	Supabase Storage	Object storage	No	No	Yes	Yes	Yes	File/audio storage
Integration	Data Cloud export layer	Export subsystem	Partial	Partial	N/A	Possible	Yes	Customer warehouse export
Integration	Snowflake connector	Warehouse connector	No	No	N/A	Possible	Yes	Customer export target

Category	Tool / Service	Type	Offline	Open Source	Free Tier / Trial	Paid Now	Paid Later	Primary Use
Integration	BigQuery connector	Warehouse connector	No	No	N/A	Possible	Yes	Customer export target
Integration	Databricks connector	Warehouse connector	No	No	N/A	Possible	Yes	Customer export target
Integration	Amazon S3 / Redshift connectors	Warehouse connectors	No	No	N/A	Possible	Yes	Customer export target
Infra	Docker Compose	Local orchestration	Yes	Yes	N/A	No	No	Local environment startup
Infra	Docker	Container platform	Yes	Yes	N/A	No	No	Service packaging
Infra	Railway	Managed hosting	No	No	Yes	Yes	Yes	Phase 1/2 hosting
Infra	AWS ECS / Fargate	Planned hosting platform	No	No	No	No	Yes	Phase 3 hosting
Infra	GitHub Actions	CI/CD platform	No	No	Yes	Possible	Yes	Build/test/deploy automation
Security	Doppler	Secrets management	No	No	Yes	Yes	Yes	Central secrets governance
Security / Edge	Cloudflare	CDN/WAF/rate limiting	No	No	Yes	Yes	Yes	Edge security and delivery
Observability	Sentry	Error monitoring	No	Partial	Yes	Possible	Yes	Error tracking
Observability	Better Stack	Logs/uptime monitoring	No	No	Yes	Possible	Yes	Operational visibility
Observability	Grafana	Metrics dashboards	Yes	Yes	Yes	Possible	Yes	Metrics and alerting
Incident Response	PagerDuty (or equivalent)	Incident tooling	No	No	Yes	No	Yes	Escalation workflow
Security	HMAC webhook verification	Security control	Yes	Partial	N/A	No	No	Webhook authenticity

Category	Tool / Service	Type	Offline	Open Source	Free Tier / Trial	Paid Now	Paid Later	Primary Use
Integration	Salesforce	CRM integration	No	No	No	Possible	Yes	CRM data sync
Integration	HubSpot	CRM integration	No	No	No	Possible	Yes	CRM data sync
Integration	Microsoft Dynamics 365	CRM integration	No	No	No	Possible	Yes	CRM data sync
Integration	Zoom	Meeting integration	No	No	Yes	Possible	Yes	Meeting ingestion
Integration	Google Meet	Meeting integration	No	No	Yes	Possible	Yes	Meeting ingestion
Integration	Microsoft Teams	Meeting integration	No	No	Yes	Possible	Yes	Meeting ingestion
Integration	Dialers / telephony tools	Telephony integrations	No	No	Possible	Possible	Yes	Call data ingestion
Integration	Gmail	Email integration	No	No	Yes	Possible	Yes	Email workflow integration
Integration	Outlook / Office 365	Email integration	No	No	Yes	Possible	Yes	Email workflow integration
Integration	Slack	Messaging integration	No	No	Yes	Possible	Yes	Notifications and reminders
Integration	LinkedIn Sales Navigator	External integration	No	No	No	Possible	Yes	Sales context enrichment
Integration	Billing provider (TBD)	Billing integration	No	No	N/A	No	Yes	Subscription events

Category	Tool / Service	Type	Offline	Open Source	Free Tier / Trial	Paid Now	Paid Later	Primary Use
Testing	pytest	Python test framework	Yes	Yes	N/A	No	No	Python AI tests
Testing	Supertest	API test library	Yes	Yes	N/A	No	No	HTTP integration tests
Testing	Testcontainers	Integration infra tool	Yes	Yes	N/A	No	Possible	Realistic infra tests
Testing	Prisma + Zod + RLS validation scripts	Validation scripts/toolchain	Yes	Partial	N/A	No	No	Contract and tenancy checks
Testing	k6	Load/performance tool	Yes	Yes	N/A	No	Possible	Performance validation
Testing	Golden datasets	Evaluation asset	Yes	Partial	N/A	No	No	AI quality regression
Testing	Prompt versioning	Governance practice	Yes	Partial	N/A	No	No	Prompt change tracking
Testing	Confidence-score gating	AI safety control	Yes	Partial	N/A	No	No	Output safety routing
Testing	Human review workflows	QA process	Yes	Partial	N/A	No	Possible	High-risk output review
AI / ML	LangChain	Library	Yes	Yes	N/A	No	No	AI Orchestration
AI / ML	Groq (Llama 3.3 70B)	Hosted inference	No	No	Yes	No	Possible	High-performance AI
AI / ML	Google Gemini (gemini-pro)	Hosted AI API	No	No	Yes	No	Possible	Multimodal AI
AI / ML	Kokoro TTS	TTS model/library	Yes	Yes	N/A	No	Possible	Text-to-Speech
AI / ML	whisper.cpp	Local runtime	Yes	Yes	N/A	No	No	Local transcription
AI / ML	CatBoost	ML library	Yes	Yes	N/A	No	Possible	Tabular ML

Category	Tool / Service	Type	Offline	Open Source	Free Tier / Trial	Paid Now	Paid Later	Primary Use
AI / ML	SHAP	ML explainability	Yes	Yes	N/A	No	No	Model explainability
AI / ML	ARIMA	Stats library	Yes	Yes	N/A	No	No	Time-series forecasting
AI / ML	CrewAI	Agent framework	Yes	Yes	N/A	No	Possible	Multi-agent orchestration
AI / ML	PyTorch	ML framework	Yes	Yes	N/A	No	No	Deep learning foundation
AI / ML	Transformers.js	Library	Yes	Yes	N/A	No	No	Browser-side AI
Frontend	Axios	Library	Yes	Yes	N/A	No	No	HTTP Client
Frontend	Lucide React	Icon library	Yes	Yes	N/A	No	No	UI Icons
Frontend	Framer Motion	Animation library	Yes	Yes	N/A	No	No	UI Animations
Frontend	SheetJS	Library	Yes	Yes	N/A	No	No	Spreadsheet processing
Frontend	jsPDF / html2canvas	Library	Yes	Yes	N/A	No	No	PDF Generation
Backend	Socket.io / WebSocket	Library	Yes	Yes	N/A	No	No	Real-time communication
Backend	Multer	Library	Yes	Yes	N/A	No	No	File upload
Backend	Nodemailer	Library	Yes	Yes	N/A	No	No	Email delivery
Backend	Bcrypt	Library	Yes	Yes	N/A	No	No	Security
AI / ML	SQLAlchemy / Alembic	ORM / Migrations	Yes	Yes	N/A	No	No	Python DB access
AI / ML	Pydantic	Data validation	Yes	Yes	N/A	No	No	Schema enforcement
AI / ML	Pandas / NumPy	Data processing	Yes	Yes	N/A	No	No	Data science utility
Integration	SendGrid	Email service	No	No	Yes	Yes	Yes	Email delivery

Category	Tool / Service	Type	Offline	Open Source	Free Tier / Trial	Paid Now	Paid Later	Primary Use
Integration	Clearbit / Bombora	Intelligence APIs	No	No	No	No	Yes	Sales Intelligence
Integration	Gong / Chorus	Conversation Intelligence	No	No	No	No	Yes	Data source
Integration	HubSpot API	CRM API	No	No	Yes	Yes	Yes	CRM Sync
Utilities	compromise / fuse.js / natural	NLP Libraries	Yes	Yes	N/A	No	No	Deterministic NLP
Utilities	python-docx / pdfkit / Jinja2	Generation tools	Yes	Yes	N/A	No	No	Document generation

8. Free Offline AI Tools

This section lists AI tools that can run locally on a developer machine, workstation, or internal server without depending on a cloud API for every request. These tools are extremely valuable during development because they help reduce API cost, support private experimentation, improve resilience, and give the team a fallback path when external services are unavailable. Our architecture already supports self-hosted or local-first thinking in important areas such as transcription and planned fallback inference, so offline AI should be treated as a serious part of the development toolbox, not just a side experiment.

We mainly use offline AI tools for:

- prompt experimentation
- local prototyping
- private testing with sample data
- fallback workflow design
- AI quality comparison
- internal demos
- early feature exploration before paying for high-usage cloud APIs

8.1 Local LLM Runtimes

Local LLM runtimes help us run AI models directly on local hardware or internal machines. These runtimes are especially useful because they provide a developer-friendly way to test AI behavior without immediately depending on external APIs.

Ollama

- **Type:** Local AI runtime
- **Offline:** Yes
- **Open Source:** Partial ecosystem / open-model friendly runtime approach
- **Why we use it:** Ollama is one of the easiest ways to run open models locally and expose them through a simple developer-friendly interface. It is useful for quick experimentation, prompt testing,

fallback design, and private internal demos. The Ollama site explicitly positions it as an easy way to build with open models and states that it can run entirely offline for mission-critical work. [web:26]

- **Use cases in development:**
 - local summarization experiments
 - testing prompt formats before paying for cloud LLM usage
 - validating whether an open model is “good enough” for a task
 - creating backup or fallback flows for AI features
 - internal demos without sending data to cloud providers
- **Why it matters for this project:** Our architecture already plans a self-hosted fallback model path and uses a provider-routing layer, so Ollama is a very practical fit for evaluation and resilience planning.

LM Studio

- **Type:** Local desktop AI runtime and local API server
- **Offline:** Yes
- **Open Source:** No, but works with open/local models
- **Why we use it:** LM Studio is very useful when we want a friendly desktop interface for local models, especially for product people, QA, freshers, or engineers who want to test prompts without setting up backend infrastructure. Its docs state that it can run local models, provide OpenAI-compatible local endpoints, and support chat with documents entirely offline. [page:1]
- **Use cases in development:**
 - easy local chat with open models
 - document Q&A on local files without cloud upload
 - serving a local OpenAI-like API for quick app experiments
 - prompt comparison between different local models
 - enabling non-backend team members to test model quality
- **Why it matters for this project:** It lowers the entry barrier for local AI testing and helps us validate user-facing AI tasks such as summaries, Q&A, and draft generation before deeper integration. [page:1]

llama.cpp-compatible runtimes

- **Type:** Local inference runtime family
- **Offline:** Yes
- **Open Source:** Yes in the broader ecosystem
- **Why we use it:** These runtimes are useful when we want efficient local execution of quantized models, especially on CPU or lighter hardware. LM Studio itself highlights support for llama.cpp and GGUF-based model execution, which makes this part of the practical local stack even if we do not use raw llama.cpp directly every day. [page:1]
- **Use cases in development:**
 - lightweight local model experiments
 - testing quantized models on limited hardware
 - evaluating CPU-friendly fallback options
 - running smaller internal assistants without API usage
- **Why it matters for this project:** Useful for low-cost experimentation and for understanding what can realistically run on developer laptops during prototyping. [page:1]

8.2 Local Open Models

Local open models are the actual AI models we can download and run through runtimes such as Ollama or LM Studio. These models are useful when we want to compare quality, cost, latency, and privacy trade-offs before committing to cloud providers for every feature.

Gemma family

- **Type:** Open model family
- **Offline:** Yes
- **Open Source:** Open model family

- **Why we use it:** Google documents Gemma as a family of open models with multiple sizes and capabilities, including deployment as-is or tuning for specific tasks. That makes Gemma a strong candidate for local experimentation, especially when we want open-model options for summarization, assistant tasks, or custom internal workflows. [page:2]
- **Use cases in development:**
 - local chat and summarization experiments
 - testing small vs medium model quality trade-offs
 - evaluating whether open models can support internal tools
 - tuning experiments for narrow internal tasks
- **Why it matters for this project:** It gives us a serious open-model path for comparing against paid APIs and helps reduce blind dependence on a single provider. [page:2]

Gemma 3

- **Type:** Open local model family
- **Offline:** Yes
- **Open Source:** Open model family
- **Why we use it:** Google lists Gemma 3 variants across sizes from lightweight text models up to larger text-and-image capable models, making it suitable for laptop experiments as well as higher-end local setups. [page:2]
- **Use cases in development:**
 - text generation and summarization
 - lightweight Q&A prototypes
 - initial multimodal testing for image-plus-text workflows
 - comparing response quality against cloud models
- **Why it matters for this project:** Good candidate for lower-cost experiments around summaries, retrieval answers, and internal copilots without immediate API expense. [page:2]

Gemma 4

- **Type:** Latest open multimodal model family
- **Offline:** Yes
- **Open Source:** Open model family
- **Why we use it:** Google describes Gemma 4 as the latest generation and notes multimodal support across text, image, and audio inputs with variants intended for mobile devices, laptops, desktops, and larger servers. This makes it especially interesting for future experiments that combine conversation intelligence with richer context sources. [page:2]
- **Use cases in development:**
 - multimodal prototyping
 - testing future workflows involving text, images, and audio
 - comparing new-generation open models against paid providers
 - evaluating whether local models can support richer assistant tasks
- **Why it matters for this project:** It is relevant because our platform works with conversations, transcripts, and related business context, so multimodal open models may become useful in advanced workflows later. [page:2]

Other practical open-model families

- **Examples:** Llama-family, Qwen, Mistral, DeepSeek, Phi
- **Why we use them:** LM Studio specifically mentions support for Llama, Qwen, and DeepSeek-style local workflows, which makes these useful comparison candidates during development and evaluation. [page:1]
- **Use cases in development:**
 - compare model quality by task
 - benchmark latency on local hardware
 - choose best model family for local fallback
 - test coding, summarization, extraction, or reasoning behavior

- **Why it matters for this project:** Different AI features need different strengths, so comparing model families helps us choose practical fallback and experimentation options. [page:1]
- **Starter recommendation:** For most prompt experiments, start with **Gemma 3** or **Llama 3.1** on Ollama because both are widely supported and easy to run.

8.3 Local Coding Assistants

Local coding assistants are tools or model setups that help with code generation, code explanation, refactoring, test creation, and debugging while keeping the workflow on local or controlled infrastructure.

Ollama with coding models

- **Type:** Local coding assistant setup
- **Offline:** Yes
- **Why we use it:** Ollama can run open coding-capable models locally, which makes it useful for code suggestions, refactoring ideas, test generation, and documentation support without sending code to external hosted tools. [web:26]
- **Use cases in development:**
 - generate boilerplate code
 - explain unfamiliar code for freshers
 - generate tests and fixtures
 - refactor repeated logic
 - help write API or prompt documentation
- **Why it matters for this project:** Useful in a mixed senior-junior team because it helps developers move faster while keeping sensitive code local.

LM Studio local coding workflows

- **Type:** Local coding assistant interface
- **Offline:** Yes
- **Why we use it:** LM Studio supports local model APIs and developer tooling, which means it can be used to test coding-focused local assistants in a safer and more controlled way than using random online tools. [page:1]
- **Use cases in development:**
 - local code explanation
 - debugging assistant for developers
 - experimenting with local code completion or review-style prompts
 - creating sample code or migration helpers
- **Why it matters for this project:** Helpful for internal engineering support and for training newer engineers without adding immediate vendor cost. [page:1]

Local code-oriented models

- **Examples:** CodeGemma and other code-tuned open models
- **Why we use them:** Google lists coding-specific variants in the Gemma family, which makes them useful candidates when we want local code-generation or code-explanation support. [page:2]
- **Use cases in development:**
 - generate starter code
 - assist in writing tests
 - explain backend or frontend patterns
 - help with boilerplate around schemas, DTOs, prompts, or handlers
- **Why it matters for this project:** Good fit for speeding up repetitive development tasks while keeping local control of the workflow. [page:2]

8.4 Local Embedding / RAG Tools

Local embedding and RAG tools help us test retrieval-based workflows without depending entirely on cloud embedding APIs. This is useful for document Q&A, search experiments, offline assistant flows, and private retrieval pipelines.

LM Studio document chat

- **Type:** Local document RAG capability
- **Offline:** Yes
- **Why we use it:** LM Studio explicitly supports chatting with documents entirely offline, which makes it one of the easiest ways to test local RAG behavior without building a full internal retrieval stack first. [page:1]
- **Use cases in development:**
 - quick document Q&A experiments
 - validating chunking and retrieval ideas
 - internal review of feature docs or architecture docs
 - testing whether local models can answer from provided business context
- **Why it matters for this project:** Our product includes retrieval-style AI features, so fast offline RAG experiments help de-risk future design work. [page:1]

pgvector in local PostgreSQL

- **Type:** Local vector storage
- **Offline:** Yes
- **Open Source:** Yes
- **Why we use it:** The architecture already uses pgvector as the planned vector store, so using it locally is the best way to test embeddings, semantic retrieval, and RAG behavior without introducing extra complexity.
- **Use cases in development:**
 - store embeddings for transcript chunks
 - test semantic search
 - prototype Ask Anything style retrieval
 - compare retrieval quality with different embedding models
- **Why it matters for this project:** Keeps local experiments aligned with the actual planned architecture.

Local embedding models

- **Type:** Local embedding inference
- **Offline:** Yes
- **Why we use them:** Local embedding models help us test semantic search and retrieval without paying for every embedding call during early experiments.
- **Use cases in development:**
 - transcript chunk embedding
 - semantic search experiments
 - RAG evaluation
 - local indexing tests
- **Why it matters for this project:** Useful for early testing and for privacy-sensitive internal experiments before committing to a hosted embedding workflow.

8.5 Local Speech and Audio Tools

Local speech and audio tools are very important for this platform because transcription and conversation analysis are a major part of the product value. Self-hosted speech tools help reduce dependency on external APIs and support privacy-sensitive testing.

Whisper

- **Type:** Local speech-to-text model
- **Offline:** Yes

- **Open Source:** Yes
- **Why we use it:** The architecture defines Whisper as the primary speech-to-text path, and explicitly notes the value of self-hosting for cost control, privacy, and independence from third-party API queue times.
- **Use cases in development:**
 - local transcription of sample calls
 - testing end-to-end ingestion and transcript flows
 - validating summary pipelines with local transcripts
 - fallback speech workflow development
- **Why it matters for this project:** Transcription is one of the most critical parts of the product, so a strong self-hosted option is strategically important.

pyannote.audio

- **Type:** Local speaker diarization tool
- **Offline:** Yes
- **Open Source:** Yes
- **Why we use it:** The architecture uses pyannote for speaker diarization, which is necessary to identify who spoke when in conversation analysis pipelines.
- **Use cases in development:**
 - speaker segmentation
 - conversation structure analysis
 - improved transcript labeling
 - testing diarization quality before production rollout
- **Why it matters for this project:** Accurate speaker separation improves transcript usefulness and downstream AI analysis.

FFmpeg and local audio preprocessing tools

- **Type:** Local media preprocessing tools
- **Offline:** Yes
- **Why we use them:** Audio often needs normalization, format conversion, clipping, or cleaning before transcription and analysis.
- **Use cases in development:**
 - converting audio formats
 - extracting audio from recordings
 - preparing files for transcription tests
 - trimming and cleaning test inputs
- **Why it matters for this project:** Makes audio pipelines more reliable during both development and QA.

8.6 Recommended Offline Use Cases

Offline AI is most useful when we want speed, privacy, low cost, or independence from external providers. It is especially strong for development and internal experimentation, but it must still be evaluated carefully before being used in customer-facing production workflows.

Recommended offline use cases for this project:

- **Prompt testing:** Try prompt formats locally before sending large volumes to paid APIs.
- **Prototype summaries:** Test whether local models can produce acceptable summaries or insights for internal review.
- **Private transcript experiments:** Run sample call analysis without sending data outside controlled environments.
- **Fallback planning:** Design backup paths for key AI features if cloud APIs are degraded or unavailable.
- **Document Q&A experiments:** Use offline RAG tools to explore retrieval-based workflows for feature ideation.

- **Junior developer support:** Let freshers use local coding assistants for explanation, refactoring help, and test generation.
- **Speech pipeline testing:** Use Whisper and diarization locally to validate end-to-end ingestion and transcription flows.

When to prefer offline AI:

- early experiments
- internal demos
- local debugging
- privacy-sensitive development
- provider fallback design
- cost-controlled prototyping

When not to assume offline AI is enough:

- if quality is below required user-facing standards
- if latency is too slow on available hardware
- if model size exceeds practical local limits
- if production operations would become harder than using a managed service

9. Free Online AI Tools and APIs

This section lists free or free-access online AI tools that are useful for experimentation, evaluation, and low-risk prototyping. These tools are valuable because they allow fast access to models and hosted experiences without local setup, but they should be treated carefully. Free online services can change quotas, limits, data handling rules, and reliability at any time, so they are best used for comparison and early-stage exploration rather than blind production dependence. [web:20][web:23]

9.0 Quick reference (short form)

Tool	Recommended daily use	Warning
Google AI Studio	Fast prompt tests and manual output comparison	Medium risk: online provider
Groq	Latency benchmarking and API prototype tests	Medium risk: online provider
Hugging Face hosted inference	Broad model-family comparisons	Medium risk: online provider
xAI / Grok routes	Alternative provider evaluation only	High risk: requires explicit approval for code use

Use this quick table for day-to-day decisions. Read the detailed subsections below when evaluating production impact.

9.1 Free Chat / Research Tools

These tools are useful when we want fast idea validation, prompt comparison, research support, or quick manual testing of how different model families behave.

Google AI Studio

- **Type:** Free online AI playground / development environment
- **Offline:** No
- **Why we use it:** Google AI Studio is useful for fast prompt testing, model evaluation, and trying multimodal or structured-output use cases without building full app integration first. It is especially

helpful during early product exploration when we want to test tasks quickly. [web:22][web:20]

- **Use cases in development:**
 - manual prompt testing
 - comparing model outputs for summaries or extraction
 - trying long-context behavior
 - evaluating prototypes before writing backend code
- **Why it matters for this project:** Helps us validate AI task quality quickly before spending engineering effort integrating a provider. [web:22]

Free research/chat tools from major providers

- **Type:** Hosted chat interfaces
- **Offline:** No
- **Why we use them:** They are useful for brainstorming, prompt drafting, requirements clarification, copy improvement, or comparing output quality manually.
- **Use cases in development:**
 - drafting prompts
 - comparing response style
 - getting ideas for feature behavior
 - manual sanity checks before implementation
- **Why it matters for this project:** Fast ideation tool, but not a replacement for governed architecture decisions.

Important rule:

- Use these tools for low-risk ideation only unless explicitly approved.
- Do not paste sensitive customer data into general-purpose free chat tools.

9.2 Free AI APIs

Free AI APIs help us test real programmatic integration without immediately paying for every request. These are useful for proof-of-concept development, benchmarking, or evaluating alternatives to primary providers.

Groq

- **Type:** Hosted inference API
- **Offline:** No
- **Why we use it:** Groq is useful when we want to test fast inference with open models and compare latency-sensitive AI workflows such as quick extraction, rapid chat, or lightweight structured tasks. Industry coverage in 2026 highlights it as one of the notable free-tier or developer-friendly inference options. [web:20][web:21]
- **Use cases in development:**
 - low-latency prototype APIs
 - speed benchmarking
 - comparing open-model responses
 - testing fallback candidates
- **Why it matters for this project:** Helpful for evaluating whether some AI tasks can run on lower-cost or faster providers than the primary stack. [web:20][web:21]

Hugging Face hosted inference options

- **Type:** Hosted model API / platform
- **Offline:** No
- **Why we use it:** Hugging Face is useful because it gives access to many model families for testing different tasks without needing to self-host each one. Their pricing page confirms they support hosted offerings and free-to-paid progression, making them suitable for experimentation and model discovery. [page:3]
- **Use cases in development:**

- trying summarization or classification models
- testing niche NLP tasks
- model benchmarking
- quick API-based proof of concept
- **Why it matters for this project:** Useful for broad experimentation before narrowing down to approved providers. [page:3]

xAI / Grok API routes (evaluation only)

- **Type:** Hosted AI API / access route
- **Offline:** No
- **Status:** Evaluation only; not approved as a default implementation dependency.
- **Why we may use it:** Useful for controlled evaluation if we want to compare alternative model families, reasoning behavior, or provider ecosystems.
- **Use cases in development:**
 - compare response quality
 - benchmark alternative providers
 - test whether a different model family fits a feature better
- **Why it matters for this project:** Comparison helps avoid over-dependence on one provider, but this should remain controlled and reviewed before any production use. [web:25][web:31]

Firm rule:

- Any xAI / Grok API usage in application code requires explicit Tech Lead approval before implementation.

Important rule:

- Any free AI API used in code must still be documented, rate-limited where needed, and reviewed for privacy and reliability before it becomes a dependency.

9.3 Free Model Hubs

Free model hubs are important because they help us discover, compare, download, and evaluate models before we decide how to serve them.

Hugging Face Hub

- **Type:** Model hub
- **Offline:** No for discovery, yes for downloaded models later
- **Why we use it:** The Gemma getting-started documentation itself points developers to Hugging Face as one place to obtain model artifacts, which shows how central model hubs are to practical open-model workflows. [page:2]
- **Use cases in development:**
 - browse available models
 - compare model families
 - download weights for local testing
 - check ecosystem support and formats
- **Why it matters for this project:** Essential for open-model evaluation, especially when testing local or fallback options. [page:2]

Kaggle model distribution path for Gemma

- **Type:** Model access platform
- **Offline:** No for access, yes after download
- **Why we use it:** Google lists Kaggle as one of the official ways to access Gemma models, making it a practical part of the open-model experimentation workflow. [page:2]
- **Use cases in development:**
 - obtain official model assets

- explore notebooks and examples
- try fine-tuning and evaluation workflows
- **Why it matters for this project:** Useful for experimentation and learning when exploring open-model alternatives. [page:2]

9.4 Best Online Tools for Prototyping

The best online tools for prototyping are the ones that reduce setup time while still helping us answer a specific technical question. We should choose them based on what we are testing, not based on hype.

Recommended online prototyping choices:

- **Google AI Studio** for prompt design, fast output comparison, and quick model experimentation. [web:22]
- **Groq** for latency testing and fast API experiments with open models. [web:20][web:21]
- **Hugging Face hosted workflows** for trying many model types quickly. [page:3]
- **Open model hubs plus local download workflows** when we want to move from online discovery to offline testing. [page:2]

How to choose the right online prototype tool:

- use **Google AI Studio** if you want quick prompt and UX experiments
- use **Groq** if latency is the main thing you want to measure
- use **Hugging Face** if you want broad model discovery and task comparison
- use **local runtimes after discovery** if privacy, repeatability, or cost control matters more than convenience

9.5 Risks and Limits of Free Online Tools

Free online AI tools are useful, but they must be treated with discipline.

Main risks:

- quotas may change without notice
- response quality may vary over time
- data retention and privacy policies may not fit customer-sensitive use
- providers may throttle or limit throughput
- free tiers may not support production-grade SLAs
- teams may accidentally build core workflows on unstable free access
- hidden migration work appears later if prototypes are not abstracted cleanly

Why this matters for our project:

- we work with customer interactions, transcripts, CRM-linked workflows, and AI-generated outputs
- many platform features are business-critical
- architecture decisions need stable, reviewed dependencies rather than accidental provider sprawl

Practical rules:

- do not treat free online AI as automatically approved for production
- do not send sensitive tenant data without review
- document every API we experiment with if it touches code or workflows
- use abstraction layers where possible so switching providers is easier
- keep provider experiments separated from core business logic

9.6 Recommended Online Use Cases

Free online AI is best when we want convenience, quick comparison, and low-friction access to models.

Recommended online use cases for this project:

- **Prompt research:** Compare how different providers respond before coding a feature.
- **Provider evaluation:** Test whether an alternative API is good enough for summaries, extraction, or Q&A.
- **Latency benchmarking:** Measure whether a provider is fast enough for user-facing tasks.
- **Rapid prototyping:** Validate feature ideas before investing in backend integration work.
- **Model discovery:** Explore model families before deciding what to download locally or pay for in production.
- **Structured output experiments:** Test JSON extraction or classification tasks quickly through hosted APIs.
- **UX exploration:** Let product and AI teams manually inspect output quality before implementation.

When to prefer free online tools:

- early exploration
- manual comparison
- non-sensitive prototyping
- speed-first evaluation
- lightweight benchmark work

When not to rely on free online tools:

- for long-term production-critical workflows
- for sensitive customer data without approval
- for anything requiring strict SLA or compliance guarantees
- when quota instability would block the feature

10. Frontend Tools

This section documents the approved frontend stack for R-Revenue Intelligence. The architecture explicitly defines the frontend as a Next.js application written in TypeScript, using TailwindCSS, shadcn/ui, TanStack Query, and Zustand, and it also defines strict rules about what the frontend should and should not do. The frontend is responsible for UI rendering, server-state display, client-state handling, and real-time user interaction, but it must not contain business logic, direct database access, or direct AI/model calls.

Why this matters in development:

- it keeps the frontend simple and maintainable
- it prevents business logic from leaking into UI code
- it makes onboarding easier for freshers and interns
- it keeps the architecture aligned with the modular backend and AI services model
- it helps us scale features without rewriting the frontend foundation

10.1 Frameworks

Next.js 15

- **Type:** Frontend framework
- **Why we use it:** The architecture defines Next.js 15 as the frontend framework for the product, with App Router support, server-side rendering for initial loads, and client-side navigation for the application experience.
- **Primary use:** Build the main web application.
- **Key use cases in development:**
 - dashboard pages
 - routed application layouts
 - authenticated workspace flows
 - server-rendered initial screens

- fast page transitions in the product UI
- **Why it matters:** It gives us a strong production-ready React foundation without having to assemble routing, rendering, and app structure from scratch.

TypeScript

- **Type:** Frontend language / typing layer
- **Why we use it:** The architecture defines TypeScript across the frontend to ensure type safety across props, API responses, and state shapes. This is especially important in a product that has many modules, AI outputs, and integration-driven UI states.
- **Primary use:** Prevent frontend bugs through static typing.
- **Key use cases in development:**
 - typed API response handling
 - safer component props
 - typed state stores
 - better refactoring support
 - easier onboarding for larger teams
- **Why it matters:** It reduces runtime mistakes and makes code easier to understand and maintain, especially for mixed-experience teams.

10.2 UI Libraries

Tailwind CSS

- **Type:** Utility-first styling framework
- **Why we use it:** The architecture explicitly states that TailwindCSS is used for all styling via utility classes and that custom CSS files should not be the normal path.
- **Primary use:** Build consistent UI quickly.
- **Key use cases in development:**
 - layout styling
 - spacing and typography
 - component states
 - responsive dashboards
 - design consistency across modules
- **Why it matters:** It speeds up UI work, reduces CSS sprawl, and makes it easier for juniors to follow a predictable styling system.

shadcn/ui

- **Type:** Component system built on Radix UI + Tailwind
- **Why we use it:** The architecture defines shadcn/ui as the chosen accessible component layer for the frontend instead of building a custom component library from scratch.
- **Primary use:** Reusable accessible UI components.
- **Key use cases in development:**
 - dialogs
 - tables
 - forms
 - popovers
 - dropdowns
 - command palettes
- **Why it matters:** It gives us modern, accessible components with low setup overhead and fits nicely with Tailwind-based styling.

Radix UI

- **Type:** Accessibility-focused primitive UI library

- **Why we use it:** The architecture specifically mentions shadcn/ui as Radix-based, which means Radix primitives are part of the practical UI foundation for accessibility and composable interactions.
- **Primary use:** Accessible low-level UI behavior.
- **Key use cases in development:**
 - modal behavior
 - keyboard navigation
 - popovers and menus
 - accessible interactive states
- **Why it matters:** It lets us build polished enterprise UI without re-implementing accessibility basics ourselves.

10.3 State Management

The architecture gives very clear state-management rules: server state must use TanStack Query, client-only UI state must use Zustand, form state must use React Hook Form, and URL state should use Next.js router and search params. This separation is important because it prevents the common mistake of mixing all state types into one store.

TanStack Query

- **Type:** Server-state management library
- **Why we use it:** The architecture explicitly defines TanStack Query as the place for API data, caching, background refetching, and optimistic updates, and it clearly says it should not be used for UI-only state.
- **Primary use:** Manage API-backed server state.
- **Key use cases in development:**
 - deal lists
 - transcript data
 - dashboard metrics
 - account records
 - background refresh after updates
- **Why it matters:** It gives predictable server-state caching and reduces manual loading, refetch, and stale-state bugs.

Zustand

- **Type:** Client-state management library
- **Why we use it:** The architecture explicitly defines Zustand as the store for client-side UI state such as panels, filters, sidebars, and selected rows.
- **Primary use:** Manage local UI state.
- **Key use cases in development:**
 - sidebar open/close
 - active tabs
 - row selection
 - temporary filters before submit
 - local interaction state
- **Why it matters:** It is lightweight, easy to understand, and prevents misuse of server-state tools for UI-only behavior.

Next.js router state

- **Type:** URL state management mechanism
- **Why we use it:** The architecture directly states that URL state such as filters and pagination should be handled using Next.js router and search params.
- **Primary use:** Persist navigable UI state in the URL.
- **Key use cases in development:**
 - pagination

- deep-linkable filters
- shareable search results
- tab routing
- **Why it matters:** URL state improves usability, refresh behavior, and collaboration because users can share the exact current view.

10.4 Forms and Validation

React Hook Form

- **Type:** Form state management library
- **Why we use it:** The architecture explicitly states that form state should be handled with React Hook Form instead of general-purpose state libraries.
- **Primary use:** Build and manage forms efficiently.
- **Key use cases in development:**
 - login forms
 - CRM connection settings
 - compliance settings
 - tracker configuration
 - search and filter forms
- **Why it matters:** It keeps forms performant, structured, and easier to validate.

Zod

- **Type:** Validation library
- **Why we use it:** Zod is defined in the architecture as the runtime validation layer for API request bodies and event payloads in the backend, and it also fits naturally with typed frontend form validation for consistent schema-driven development.
- **Primary use:** Validate data shapes consistently.
- **Key use cases in development:**
 - form schema validation
 - API payload shape checks
 - safe parsing of user input
 - shared contract thinking between frontend and backend
- **Why it matters:** It reduces invalid input bugs and creates clearer contracts between UI and services.

10.5 Charts and Visualization

The architecture mentions dashboards, forecast views, performance insights, and analytical outputs, which means charting and visualization are important frontend concerns even if the specific chart library is not hard-fixed in the core architecture document. The frontend should therefore use a simple, approved charting approach that integrates well with React and TypeScript while staying easy for juniors to maintain.

Recharts

- **Type:** React charting library
- **Why we use it:** Recharts is a practical choice for React dashboards because it is easy to integrate, readable for new engineers, and suitable for KPI cards, trend charts, and reporting widgets.
- **Primary use:** Product dashboards and analytics widgets.
- **Key use cases in development:**
 - revenue trend charts
 - call activity graphs
 - pipeline health visuals
 - coaching score breakdowns
 - forecasting summaries
- **Why it matters:** It gives fast chart implementation without making the frontend stack too complex.

Alternative rule for charts

- If we adopt another chart library later, it should still follow the core principle of keeping frontend logic simple and presentation-focused, while all calculations and business rules stay in backend or analytics layers.
- New charting libraries should not be added without review, because visual libraries can become heavy and inconsistent across the product.

10.6 Frontend Testing

Frontend testing is important because this product contains many interactive screens, filters, tables, forms, and real-time UI states. The architecture also emphasizes system-level and integration-level validation, so frontend tests must cover both UI behavior and connected flows.

Playwright

- **Type:** End-to-end browser testing tool
- **Why we use it:** Playwright is a strong fit for realistic end-to-end browser testing across authenticated flows, dashboards, forms, and multi-step user interactions. The project architecture emphasizes end-to-end system behavior and QA readiness, which makes Playwright a natural approved testing tool.
- **Primary use:** End-to-end frontend testing.
- **Key use cases in development:**
 - login and auth flows
 - CRM integration setup screens
 - dashboard behavior
 - transcript search flows
 - regression checks across major user journeys
- **Why it matters:** It tests the product the way real users use it.

Jest / component-level test utilities

- **Type:** Frontend unit and component testing tools
- **Why we use them:** Jest is already part of the broader testing direction in the stack and is useful for validating utility logic, hooks, component behavior, and transformation helpers.
- **Primary use:** Small-scope frontend correctness checks.
- **Key use cases in development:**
 - utility function tests
 - component state logic
 - hook behavior
 - rendering conditions
- **Why it matters:** It catches issues early before they become full end-to-end failures.
- **Why it matters:** It catches issues early before they become full end-to-end failures.

10.7 Specialized Frontend Utilities

Axios

- **Type:** HTTP Client
- **Why we use it:** Standard library for making HTTP requests from the frontend to the backend services.
- **Primary use:** API communication.
- **Why it matters:** Provides a consistent and feature-rich interface for handling requests and responses.

Lucide React

- **Type:** Icon library
- **Why we use it:** Provides a comprehensive set of clean, consistent icons for the enterprise UI.
- **Primary use:** UI iconography.
- **Why it matters:** Ensures a professional and unified look across the application.

Framer Motion

- **Type:** Animation library
- **Why we use it:** Used to implement smooth, premium micro-animations and transitions to enhance the user experience.
- **Primary use:** UI animations and dynamic transitions.
- **Why it matters:** Elevates the product's aesthetic quality and feel.

jsPDF and html2canvas

- **Type:** Document generation libraries
- **Why we use it:** Used to generate downloadable PDF reports from dashboard views and AI-generated insights.
- **Primary use:** Report generation.
- **Why it matters:** Allows users to export AI findings for offline use or sharing.

11. Backend Tools

This section documents the approved backend stack for R-Revenue Intelligence. The architecture defines the API layer as a NestJS modular monolith in Phase 1 and Phase 2, with TypeScript for all product services, Prisma ORM for database access, Zod for runtime validation, Supabase Auth for authentication, JWT guards for request protection, and BullMQ for internal event-driven communication. It also clearly states that backend services own business logic, CRM sync, event orchestration, and coordination with Python AI services, while AI inference itself must stay outside the backend TypeScript layer.

Why this matters in development:

- it keeps business logic in one clear place
- it prevents AI logic from leaking into product services
- it enforces module boundaries
- it makes event-driven growth easier later
- it keeps the product stack simple for a team that includes junior developers

11.1 Core Frameworks

NestJS v11

- **Type:** Backend application framework
- **Why we use it:** The architecture defines NestJS v11 as the core framework for the modular monolith, mapping backend modules to product modules and providing dependency injection, guards, and structured service organization.
- **Primary use:** Build modular backend APIs and application services.
- **Key use cases in development:**
 - module-based feature organization
 - controllers and services
 - guards and interceptors
 - event consumers and orchestrators
 - internal API coordination
- **Why it matters:** It gives a clean enterprise-friendly structure, which is very useful when the system is divided into multiple platform modules.

TypeScript

- **Type:** Backend language
- **Why we use it:** The architecture explicitly states that all product services must use TypeScript and that there are no exceptions for product feature code, APIs, event handling, CRM sync, or workflow logic.
- **Primary use:** Type-safe backend development.
- **Key use cases in development:**
 - service implementation
 - API contracts
 - DTO typing
 - event payload typing
 - integration clients
- **Why it matters:** It improves maintainability and keeps frontend and backend aligned around a shared language for product engineering.

Prisma ORM

- **Type:** ORM and migration tool
- **Why we use it:** The architecture defines Prisma ORM as the type-safe PostgreSQL access layer and migration workflow for the API layer.
- **Primary use:** Database reads, writes, and schema migrations.
- **Key use cases in development:**
 - tenant-scoped queries
 - transactional writes
 - relational data access
 - schema evolution
 - migration workflows
- **Why it matters:** It reduces raw-query errors and makes database access easier to reason about for both experienced and junior engineers.

SQLAlchemy and Alembic

- **Type:** Python ORM and migration tool
- **Why we use it:** For AI services that require persistent state or complex relational data management in the Python layer.
- **Primary use:** AI service data persistence.
- **Key use cases:**
 - Storing AI insights and intermediate deal analytics
 - Managing schema migrations for AI-specific data
- **Why it matters:** Standardizes database access in the Python/FastAPI layer.

11.2 API and Validation

REST API pattern

- **Type:** API design approach
- **Why we use it:** The frontend is documented to connect to the backend through REST APIs, and the API layer is responsible for all reads and writes between UI and product services.
- **Primary use:** Frontend-to-backend communication.
- **Key use cases in development:**
 - CRUD operations
 - fetch module data
 - create async jobs
 - update settings
 - read summaries and reports
- **Why it matters:** It keeps the product interface predictable and easy to test.

- **Type:** Runtime validation library
- **Why we use it:** The architecture states that Zod is used for runtime validation of all API request bodies and event payloads and that no unvalidated request should reach business logic.
- **Primary use:** Validate API and event contracts.
- **Key use cases in development:**
 - request payload validation
 - response-shape confidence
 - event payload checking
 - internal service contract enforcement
- **Why it matters:** It protects the system from bad input and keeps contracts explicit.

Swagger / API documentation style

- **Type:** API documentation support
- **Why we use it:** Even though the architecture document does not deeply detail Swagger setup in the excerpt, an API-heavy modular platform benefits from clear self-documented endpoints and contract visibility.
- **Primary use:** API discoverability and testing support.
- **Key use cases in development:**
 - internal endpoint review
 - QA collaboration
 - frontend-backend alignment
 - faster debugging
- **Why it matters:** It helps freshers and cross-functional teams understand available APIs without reading backend code line by line.

11.3 Authentication and Authorization

Supabase Auth

- **Type:** Managed authentication service
- **Why we use it:** The architecture defines Supabase Auth as the authentication layer for sign-up, sign-in, OAuth, session management, and JWT issuance, and it lists it among the key external dependencies and critical platform services.
- **Primary use:** User authentication and identity flow management.
- **Key use cases in development:**
 - login and signup
 - session handling
 - OAuth-based identity flows
 - token issuance for protected APIs
- **Why it matters:** It removes the need to build a custom auth server during early phases and helps the team ship faster.

JWT Guards

- **Type:** Authorization enforcement mechanism
- **Why we use it:** The architecture explicitly states that JWT guards verify JWTs and enforce RBAC on every request and that no route should be left unguarded.
- **Primary use:** Secure backend endpoints.
- **Key use cases in development:**
 - route protection
 - role-based access
 - tenant-aware request validation
 - secure module APIs
- **Why it matters:** It makes authorization enforcement explicit and consistent.

RBAC

- **Type:** Role-based access control model
- **Why we use it:** The architecture describes RBAC as part of the platform-wide governance and ties users such as AE, SDR, Sales Manager, VP Sales, CRO, and RevOps to specific access roles and module usage patterns.
- **Primary use:** Restrict functionality by role.
- **Key use cases in development:**
 - manager-only features
 - admin settings access
 - role-specific dashboards
 - workflow permissions
- **Why it matters:** Revenue tools often expose sensitive data, so role boundaries must be clear from day one.

11.4 Queue and Event Tools

The architecture makes event-driven communication a core principle. It clearly states that modules must communicate via events rather than direct synchronous internal calls, with BullMQ and Redis acting as the internal event-bus and queueing backbone. This is very important because the system depends on async transcription, AI workflows, retries, and inter-module processing.

BullMQ

- **Type:** Queue and event orchestration library
- **Why we use it:** The architecture explicitly defines BullMQ as the internal event bus between modules and requires AI calls and inter-module async workflows to use queue-based patterns. It also requires events to be idempotent because retries can happen.
- **Primary use:** Background jobs and event-driven module communication.
- **Key use cases in development:**
 - transcription pipeline jobs
 - AI workflow scheduling
 - event publishing and consumption
 - retries and dead-letter handling
 - debounce and deduplication patterns
- **Why it matters:** It helps decouple modules and makes heavy or slow operations safe to run asynchronously.

Redis

- **Type:** Queue backing store and cache
- **Why we use it:** The architecture identifies Redis as the backing store for BullMQ and also highlights it as a critical path dependency because if Redis goes down, event processing and queue-based AI workflows stop.
- **Primary use:** Queue state, cache, and async coordination.
- **Key use cases in development:**
 - BullMQ backing
 - transient cache
 - short-lived coordination state
 - performance optimization
- **Why it matters:** It is a core operational dependency for the event-driven architecture.

Event contracts

- **Type:** Architectural communication pattern
- **Why we use it:** The architecture defines named events such as `call.transcription.completed`, `revenue_graph.entity.linked`, `tracker.detection.created`, and `deal.stage.changed`, and it states that consumers must subscribe to published events rather than directly coupling to internal module code.

- **Primary use:** Stable module-to-module communication.
- **Key use cases in development:**
 - notify downstream modules of completed work
 - trigger AI pipelines
 - chain processing across lifecycle stages
 - support retries and decoupled scaling
- **Why it matters:** It protects modularity and makes later module extraction easier.

11.5 Backend Testing

Backend testing is critical because the backend owns business logic, module boundaries, validation, integrations, security enforcement, and async workflows. The architecture also stresses end-to-end data flows, event verification, and integration correctness, which means backend testing must include more than simple unit tests.

Jest

- **Type:** Backend unit testing framework
- **Why we use it:** Jest is a standard and practical choice for testing NestJS services, utilities, guards, and event handlers in a TypeScript backend.
- **Primary use:** Unit and service-level testing.
- **Key use cases in development:**
 - service logic tests
 - guard tests
 - validation tests
 - event handler logic
- **Why it matters:** It catches product logic issues early and works well with the TypeScript backend stack.

Supertest

- **Type:** API integration testing library
- **Why we use it:** It is well suited for validating NestJS endpoints end to end at the HTTP layer, including auth, validation, status codes, and response contracts.
- **Primary use:** API endpoint testing.
- **Key use cases in development:**
 - auth-protected route tests
 - request validation tests
 - response shape verification
 - RBAC route checks
- **Why it matters:** It validates real API behavior, not just isolated functions.

Testcontainers

- **Type:** Integration test environment tool
- **Why we use it:** The broader stack direction favors realistic testing with actual infrastructure dependencies such as PostgreSQL and Redis rather than only mocks.
- **Primary use:** Run realistic integration tests.
- **Key use cases in development:**
 - database-backed service tests
 - queue integration tests
 - event flow verification
 - startup and migration checks
- **Why it matters:** It increases confidence in infrastructure-dependent backend behavior.

Backend testing principle

- Use unit tests for logic.

- Use integration tests for APIs, auth, DB, and queue behavior.
- Use event-flow tests for BullMQ-based workflows.
- Do not test AI model quality inside backend service tests; test contract handling and orchestration instead, because inference belongs to Python AI services.

12. AI and ML Stack

This section defines the approved AI and ML stack for R-Revenue Intelligence. The architecture is very clear on one core rule: all AI and ML logic belongs in Python services, while TypeScript product services only orchestrate business workflows, queues, APIs, and integrations. The platform also defines OpenAI as the primary LLM provider through LiteLLM, Whisper as the primary ASR path, AssemblyAI as the speech fallback, and pgvector-based retrieval for RAG-style features.

Why this matters in development:

- it prevents AI logic from leaking into product services
- it makes model replacement easier later
- it allows better scaling of AI-heavy workloads
- it supports fallback and evaluation strategies
- it keeps the stack understandable even for freshers: TypeScript for product logic, Python for AI logic only

12.1 Primary LLM Providers

OpenAI API

- **Type:** Managed LLM API
- **Why we use it:** The architecture explicitly approves OpenAI API as the primary LLM provider and records this as an ADR-level decision. It is used for summaries, Q&A, email draft generation, RAG outputs, and AI-assisted workflows across modules such as M-02, M-06, and M-10.
- **Primary use:** Production-grade LLM inference.
- **Key use cases in development and product:**
 - call summaries
 - deal briefs
 - account briefs
 - AI-generated emails
 - natural-language query answering
 - trainer simulation and coaching outputs
- **Why it matters:** It is the main cloud AI capability in the approved architecture, so many downstream features depend on it.

OpenAI for structured JSON output

- **Type:** Applied LLM usage pattern
- **Why we use it:** The architecture repeatedly emphasizes that AI services should return structured JSON to TypeScript services rather than free-form product logic. This makes OpenAI useful not just for text generation but for machine-readable outputs such as next steps, risks, extracted entities, and confidence scores.
- **Primary use:** Controlled AI outputs for product workflows.
- **Key use cases in development:**
 - extraction from calls
 - next-step generation
 - answer-with-citations style responses
 - AI field suggestions for CRM enrichment
- **Why it matters:** Structured outputs are easier to validate, test, and safely integrate than raw text alone.

12.2 Free / Backup LLM Providers

The approved architecture already anticipates provider fallback and explicitly mentions LiteLLM routing with Anthropic as a fallback path, while earlier sections of this document also include local and free-access options for experimentation and backup. That means the team should think of backup providers in two layers: approved fallback for production resilience, and free/local providers for development and experimentation.

LiteLLM fallback routing

- **Type:** Provider routing strategy
- **Why we use it:** The architecture states that LiteLLM provides unified LLM gateway behavior with OpenAI as primary and Anthropic as fallback, and it also calls out the need for fallback handling in external AI dependency planning.
- **Primary use:** Provider abstraction and fallback routing.
- **Key use cases in development:**
 - switch providers without changing all calling code
 - centralize provider credentials and budgets
 - support resilience when one provider fails
 - compare outputs across providers
- **Why it matters:** It reduces vendor lock-in and helps the system fail more gracefully.

Anthropic fallback path

- **Type:** Secondary managed LLM provider
- **Why we use it:** The architecture mentions LiteLLM to Anthropic as the fallback path for OpenAI-backed LLM workflows.
- **Primary use:** Reliability backup for cloud LLM workflows.
- **Key use cases in development:**
 - provider resilience testing
 - fallback scenario validation
 - side-by-side response quality checks
- **Why it matters:** It helps ensure that core AI features do not depend on one provider only.

Ollama and local open-model fallback

- **Type:** Local LLM runtime
- **Why we use it:** While not the primary production path, Ollama is valuable for experimentation, privacy-sensitive internal tests, and offline fallback design. Its site states it supports running models locally and offline. [web:26]
- **Primary use:** Development fallback and private local AI.
- **Key use cases in development:**
 - prompt testing
 - internal demos
 - backup local assistant flows
 - cost-controlled AI experiments
- **Why it matters:** It gives us a no-cloud fallback option for early experiments and resilience planning. [web:26]

Google AI Studio / Groq / Hugging Face hosted options

- **Type:** Free or low-barrier online evaluation tools
- **Why we use them:** These are useful for model comparison and prototyping, but they should remain evaluation tools unless explicitly reviewed for production usage. [web:20][web:22][page:3]
- **Primary use:** Prototyping and model benchmarking.
- **Key use cases in development:**
 - provider comparison
 - latency benchmarking

- structured output experiments
- manual prompt testing
- **Why it matters:** They help us evaluate alternatives without immediately changing the approved production path. [web:20][web:22][page:3]

12.3 AI Frameworks and Orchestration

Python 3.12

- **Type:** AI/ML runtime language
- **Why we use it:** The architecture explicitly defines Python as the only allowed language for AI and ML services, with TypeScript reserved for product services.
- **Primary use:** Implement all AI and ML logic.
- **Key use cases in development:**
 - model inference services
 - NLP pipelines
 - embeddings
 - speech processing
 - forecasting workflows
- **Why it matters:** It creates a clean and non-negotiable boundary between product logic and AI logic.

FastAPI

- **Type:** AI service framework
- **Why we use it:** The architecture defines FastAPI as the internal HTTP API layer for AI services and for the separate transcription service. It is used to expose endpoints such as `/v1/summarize`, `/v1/answer-query`, `/v1/embed`, `/v1/score-call`, and transcription-related endpoints.
- **Primary use:** Expose internal AI APIs to product services.
- **Key use cases in development:**
 - internal LLM endpoints
 - embedding endpoints
 - summarization endpoints
 - transcript processing services
 - confidence-scored AI responses
- **Why it matters:** It keeps AI services clean, fast to build, and easy to document.

LiteLLM

- **Type:** LLM gateway and routing framework
- **Why we use it:** The architecture explicitly names LiteLLM as the unified LLM gateway, with OpenAI primary and Anthropic fallback. It also notes that API layer code must not call provider SDKs directly.
- **Primary use:** Centralized LLM access.
- **Key use cases in development:**
 - provider abstraction
 - fallback routing
 - budget control
 - easier future provider switching
- **Why it matters:** It prevents provider-specific code from spreading across the AI stack.

LangGraph

- **Type:** Agent and workflow orchestration framework
- **Why we use it:** The architecture defines LangGraph as the stateful multi-step workflow engine for features like AI Deep Researcher and Ask Anything.
- **Primary use:** Multi-step AI orchestration.
- **Key use cases in development:**
 - deep research pipelines

- multi-step retrieval and answer generation
- stateful agent workflows
- controlled reasoning flows
- **Why it matters:** It is a good fit for complex AI tasks that need more than one prompt or one model call.

BullMQ-driven async AI execution

- **Type:** Queue orchestration pattern
- **Why we use it:** The architecture explicitly says AI calls must be async through BullMQ rather than direct synchronous request-handler calls, especially for heavy or long-running tasks.
- **Primary use:** Reliable async AI processing.
- **Key use cases in development:**
 - summarization jobs
 - tracker detection
 - transcription pipelines
 - retries and delayed processing
- **Why it matters:** It improves reliability and keeps user-facing APIs responsive.

12.4 NLP Libraries

spaCy

- **Type:** NLP preprocessing library
- **Why we use it:** The architecture explicitly lists spaCy as the NLP preprocessing tool for tokenization and entity extraction and also notes that it does not perform LLM calls.
- **Primary use:** Traditional NLP preprocessing.
- **Key use cases in development:**
 - tokenization
 - named entity extraction
 - text preprocessing before AI steps
 - lightweight linguistic analysis
- **Why it matters:** Not every text problem needs an LLM, and spaCy gives us fast, structured NLP building blocks.

Topic modeling and clustering utilities

- **Type:** NLP analysis tooling category
- **Why we use them:** The architecture ties conversation intelligence features to theme detection, topic tagging, and topic modeling across call data.
- **Primary use:** Discover recurring themes and discussion topics.
- **Key use cases in development:**
 - AI Theme Spotter
 - AI Topic Tagger
 - call clustering
 - trend analysis across conversations
- **Why it matters:** These are core product features in the “Understand” stage of the platform.

Intent detection pipelines

- **Type:** NLP + semantic classification workflow
- **Why we use them:** The architecture explicitly describes AI Smart Tracker as intent-based signal detection using semantic NLP rather than simple keyword matching.
- **Primary use:** Detect sales signals from language.
- **Key use cases in development:**
 - identify objections
 - detect pricing discussions
 - find next-step commitments

- surface risk signals
- **Why it matters:** It turns raw conversation text into actionable structured signals.

12.5 Speech-to-Text and Audio

Whisper

- **Type:** ASR model / transcription engine
- **Why we use it:** Whisper is explicitly approved as the primary speech-to-text path and is identified as part of the critical revenue path in the architecture. It is used in M-01 and sits at the start of much of the downstream AI pipeline.
- **Primary use:** Convert raw audio into transcript text.
- **Key use cases in development and product:**
 - call transcription
 - meeting transcription
 - transcript generation for AI analysis
 - self-hosted or controlled ASR workflows
- **Why it matters:** No transcript means no downstream AI value for many modules.

AssemblyAI

- **Type:** Speech API / fallback ASR provider
- **Why we use it:** The architecture explicitly approves AssemblyAI as the fallback ASR path and also ties it to speaker diarization support.
- **Primary use:** Backup ASR and diarization support.
- **Key use cases in development:**
 - ASR fallback tests
 - compare transcript quality
 - speaker-labeled transcript generation
 - resilience planning
- **Why it matters:** It reduces risk if Whisper-based processing is unavailable or insufficient for a specific case.

pyannote.audio

- **Type:** Speaker diarization library
- **Why we use it:** The architecture explicitly lists pyannote in the transcription service as the tool for speaker diarization.
- **Primary use:** Identify who spoke when.
- **Key use cases in development:**
 - speaker segmentation
 - cleaner transcript structure
 - improved downstream summaries and scoring
- **Why it matters:** Better speaker labeling makes call intelligence more trustworthy.

FFmpeg and preprocessing utilities

- **Type:** Audio preprocessing tooling
- **Why we use them:** Audio workflows usually need format conversion and preprocessing before ASR.
- **Primary use:** Prepare media for transcription.
- **Key use cases in development:**
 - extract audio
 - normalize input files
 - convert formats
 - build stable test fixtures
- **Why it matters:** More stable inputs produce more reliable speech pipelines.

12.6 Embeddings and Vector Search

pgvector

- **Type:** Vector extension for PostgreSQL
- **Why we use it:** The architecture explicitly uses pgvector for semantic search and RAG, including the Ask Anything retrieval flow.
- **Primary use:** Store embeddings close to application data.
- **Key use cases in development:**
 - transcript chunk embeddings
 - semantic search
 - Ask Anything retrieval
 - context retrieval for summaries or answers
- **Why it matters:** It keeps the early vector workflow aligned with the main database and avoids premature infrastructure sprawl.

/v1/embed AI endpoint

- **Type:** Internal embedding API pattern
- **Why we use it:** The architecture explicitly defines an internal embed endpoint used by modules such as M-05 and M-06.
- **Primary use:** Standardized embedding generation.
- **Key use cases in development:**
 - embedding transcript chunks
 - embedding user queries
 - retrieval indexing
 - semantic similarity workflows
- **Why it matters:** It gives a central place to manage embedding logic instead of spreading it across modules.

sentence-transformers / local embedding models

- **Type:** Embedding model tooling
- **Why we use them:** The architecture mentions embedding generation as part of the Python AI services layer, and local embedding models are useful for testing retrieval workflows without always paying for hosted embeddings.
- **Primary use:** Local or controlled embedding generation.
- **Key use cases in development:**
 - cheap retrieval experiments
 - offline semantic search testing
 - compare embedding quality
 - private RAG experiments
- **Why it matters:** Helpful for development, evaluation, and controlled experimentation.

12.7 ML / Forecasting Libraries

The platform's forecasting module is not just an LLM feature. The architecture explicitly describes revenue projection as combining execution data with historical conversion patterns and weighted pipeline modeling, which means classical ML and analytical methods are important here.

scikit-learn

- **Type:** Classical ML library
- **Why we use it:** It is a practical choice for baseline ML workflows, feature engineering experiments, and classification/regression tasks in forecasting-related work.
- **Primary use:** Baseline ML and structured predictive modeling.
- **Key use cases in development:**

- lead or deal feature experiments
- baseline predictive models
- clustering and preprocessing pipelines
- evaluation against heuristic methods
- **Why it matters:** It helps us avoid forcing every predictive problem into an LLM-only solution.

XGBoost

- **Type:** Gradient boosting library
- **Why we use it:** For forecasting and pipeline risk scoring, boosted-tree models are often strong on tabular business data and are a practical fit for historical conversion and weighted pipeline features.
- **Primary use:** Forecasting and risk scoring on structured business data.
- **Key use cases in development:**
 - revenue prediction experiments
 - deal risk scoring
 - historical conversion modeling
 - benchmark models against simpler baselines
- **Why it matters:** Better fit than pure LLMs for many tabular prediction tasks.

Statistical forecasting logic

- **Type:** Forecasting method family
- **Why we use it:** The architecture directly describes forecasting as using historical conversion rates and weighted pipeline modeling, which means some predictions will rely on explicit business logic and statistical methods, not only ML models.
- **Primary use:** Transparent forecasting and sanity checks.
- **Key use cases in development:**
 - weighted pipeline forecasts
 - coverage metrics
 - target tracking
 - explainable revenue projections
- **Why it matters:** Forecasting needs both accuracy and explainability for business users.

12.8 AI Testing and Evaluation Tools

AI testing matters because the architecture requires structured outputs, confidence scores, fallback handling, and safe routing of flagged outputs. The system also states that low-confidence outputs should be marked for review and should not be auto-written to CRM.

Golden datasets

- **Type:** Evaluation dataset approach
- **Why we use it:** AI features should be tested against known examples instead of only ad hoc prompt checks.
- **Primary use:** Regression testing for AI behavior.
- **Key use cases in development:**
 - compare summary quality over time
 - validate tracker extraction
 - test classification consistency
 - evaluate prompt changes safely
- **Why it matters:** It helps us detect quality regressions before shipping.

Confidence-score gating

- **Type:** AI quality control pattern
- **Why we use it:** The architecture explicitly states that AI endpoints return `confidence_score` and that results below a threshold should be flagged for review.
- **Primary use:** Safe automation control.

- **Key use cases in development:**
 - hold risky outputs for human review
 - prevent bad CRM auto-writes
 - compare model confidence over versions
- **Why it matters:** It gives an operational safety layer around AI outputs.

Contract tests for AI endpoints

- **Type:** API correctness testing
- **Why we use it:** Since TypeScript services depend on structured JSON outputs from Python AI services, contract testing is essential.
- **Primary use:** Verify schema and response correctness.
- **Key use cases in development:**
 - validate JSON output shape
 - test required fields
 - test confidence-score presence
 - verify backward compatibility across `/v1` and future `/v2`
- **Why it matters:** The application depends on predictable AI outputs, not just good-looking text.

Human review workflows

- **Type:** Operational evaluation mechanism
- **Why we use it:** The architecture clearly states that flagged outputs should go to review instead of automatic CRM write-back.
- **Primary use:** Human-in-the-loop safety.
- **Key use cases in development:**
 - validate low-confidence summaries
 - inspect uncertain extracted fields
 - tune prompts with reviewer feedback
- **Why it matters:** It improves trust in the system and supports safer rollout.

12.10 Specialized AI Tools and Libraries

LangChain

- **Type:** AI orchestration framework
- **Why we use it:** LangChain is used for building complex AI chains, managing memory, and orchestrating multi-step LLM interactions where LangGraph might be too heavy or for specific sub-tasks.
- **Primary use:** Context building and prompt chain management.
- **Key use cases:**
 - Stakeholder power mapping
 - Negotiation simulation
 - Deep personalization outreach
- **Why it matters:** It provides a rich set of utilities for context management that complements LangGraph.

Groq (Llama 3.3 70B & 3.1 70B)

- **Type:** High-performance inference provider
- **Why we use it:** Groq provides ultra-fast inference for Llama models, which is critical for real-time features like chat and scoring.
- **Primary use:** Real-time AI interactions.
- **Key use cases:**
 - Real-time scoring and chat
 - Fast context extraction
 - Latency-sensitive summarization

- **Why it matters:** Speed is a feature in conversational intelligence.

Kokoro TTS

- **Type:** Text-to-Speech library/model
- **Why we use it:** Used to generate high-quality voice responses for AI coaching and simulation features.
- **Primary use:** Conversational AI voice output.
- **Key use cases:**
 - Negotiation simulator voice
 - AI coach feedback
- **Why it matters:** Improves immersion in training simulations.

scikit-learn, XGBoost, and CatBoost

- **Type:** Machine Learning libraries
- **Why we use it:** These libraries are used for predictive modeling on tabular data, which is often more efficient than using LLMs for numerical forecasting.
- **Primary use:** Win probability prediction and revenue forecasting.
- **Key use cases:**
 - Deal win probability prediction (XGBoost/CatBoost)
 - Revenue forecasting (ARIMA/scikit-learn)
 - Identifying top deal risk factors (SHAP)
- **Why it matters:** Enterprise forecasting requires high accuracy and explainability.

12.9 AI service operating depth (FastAPI, LiteLLM, LangGraph, spaCy, sentence-transformers)

These dependencies sit on critical AI paths and need explicit operating rules, not just tool descriptions.

FastAPI operating profile

- **Owner:** AI Lead
- **Responsibility:** Keep internal AI endpoint contracts stable and observable.
- **Guardrails:**
 - version endpoints when response contracts change
 - enforce request timeout and payload limits
 - expose health and readiness probes
 - return structured errors compatible with TypeScript consumers

LiteLLM operating profile

- **Owner:** AI Lead with Tech Lead oversight
- **Responsibility:** Centralize provider routing, fallback behavior, and provider policy controls.
- **Guardrails:**
 - all provider calls route through LiteLLM
 - fallback behavior is tested and documented
 - token usage and cost are monitored by provider
 - provider-specific logic remains isolated from product services

LangGraph operating profile

- **Owner:** AI Lead
- **Responsibility:** Maintain deterministic and debuggable multi-step AI workflows.
- **Guardrails:**
 - define and version workflow state schemas
 - set max-step and timeout limits per workflow
 - persist intermediate states for debugging and evaluation

- require regression tests for graph-logic changes

spaCy operating profile

- **Owner:** AI Lead
- **Responsibility:** Provide deterministic NLP preprocessing for extraction and pipeline prep.
- **Guardrails:**
 - pin package and model versions
 - validate NER/tokenization quality on representative data
 - treat outputs as preprocessing signals, not final business decisions by default

sentence-transformers operating profile

- **Owner:** AI Lead with Data Lead review
- **Responsibility:** Keep embedding quality stable across retrieval workflows.
- **Guardrails:**
 - pin embedding model versions and document migration notes
 - evaluate retrieval quality against golden datasets
 - run controlled re-indexing when embedding models change
 - monitor latency and memory behavior in production-like environments

13. Data Layer

This section defines the approved data layer for R-Revenue Intelligence. The architecture explicitly approves PostgreSQL as the primary relational database, Redis plus BullMQ for cache and event-driven processing, Meilisearch for search, ClickHouse for analytics, and Data Cloud connectors for exporting customer-owned data to external warehouses. It also enforces multi-tenancy through shared PostgreSQL with row-level security and `tenant_id` on every write.

Why this matters in development:

- it tells every engineer where each kind of data belongs
- it prevents random data-store sprawl
- it keeps local and production architecture aligned
- it supports modular separation and tenant isolation
- it makes future scaling decisions easier to reason about

13.1 Primary Database

PostgreSQL

- **Type:** Primary relational database
- **Why we use it:** PostgreSQL is explicitly approved in the architecture as the primary relational database and is captured in ADR-004. It stores users, accounts, deals, transcripts, detections, CRM sync state, and other core platform data.
- **Primary use:** System of record for transactional product data.
- **Key use cases in development:**
 - tenant-scoped business entities
 - transcript storage
 - AI output persistence
 - audit and sync records
 - feature data models
- **Why it matters:** It is the core data backbone for the entire platform.

Shared PostgreSQL with Row-Level Security

- **Type:** Multi-tenant database architecture

- **Why we use it:** The architecture explicitly defines shared PostgreSQL with RLS and `tenant_id`-based isolation as the approved multi-tenancy model and records this as ADR-008.
- **Primary use:** Tenant isolation in a shared database.
- **Key use cases in development:**
 - secure multi-tenant queries
 - prevent cross-tenant access
 - enforce ownership rules
 - simplify operational architecture for early phases
- **Why it matters:** Multi-tenancy safety is non-negotiable in this platform.

Supabase PostgreSQL

- **Type:** Managed PostgreSQL deployment option
- **Why we use it:** The architecture refers to Supabase PostgreSQL with RLS and PITR as the Phase 1 primary database setup and also uses Supabase as part of the broader managed early-stage infrastructure.
- **Primary use:** Managed relational DB in early phases.
- **Key use cases in development:**
 - fast project setup
 - managed backups
 - easier auth integration
 - lower DevOps burden for MVP
- **Why it matters:** It helps the team ship quickly before moving to more customized infra at larger scale.

13.2 Cache and Queue Storage

Redis

- **Type:** In-memory cache and queue backing store
- **Why we use it:** The architecture explicitly defines Redis as the cache layer and as the critical dependency backing BullMQ. It also calls Redis a single point of failure in early phases and highlights it as the critical path for event processing.
- **Primary use:** Cache, ephemeral state, and queue backing.
- **Key use cases in development:**
 - rate limits
 - short-lived cache
 - queue state
 - session-like temporary coordination
- **Why it matters:** If Redis fails, event-driven processing stalls.

BullMQ on Redis

- **Type:** Queue and event-bus storage pattern
- **Why we use it:** The architecture explicitly approves BullMQ on Redis as the event bus in ADR-005 and uses it for transcription jobs, AI processing, inter-module events, retries, and dead-letter behavior.
- **Primary use:** Durable async job and event handling.
- **Key use cases in development:**
 - call transcription jobs
 - AI processing jobs
 - inter-module event delivery
 - retries and delayed jobs
- **Why it matters:** It is essential for the modular event-driven design.

Upstash Redis HA

- **Type:** Scaled managed Redis option

- **Why we use it:** The architecture identifies Upstash Redis HA as a planned resilience upgrade path for Redis in later phases.
- **Primary use:** Higher-availability Redis at scale.
- **Key use cases in development and future ops:**
 - reduce Redis single-point-of-failure risk
 - support higher uptime needs
 - improve production readiness
- **Why it matters:** Important for scale and reliability planning.

13.3 Search Engine

Meilisearch

- **Type:** Search index
- **Why we use it:** The architecture explicitly lists Meilisearch as the search index for transcripts, emails, and account/deal metadata, with typo-tolerant and fast search behavior.
- **Primary use:** Product search and discovery.
- **Key use cases in development:**
 - conversation library search
 - transcript search
 - account and deal metadata lookup
 - filtered search UX
- **Why it matters:** Search is a core product capability, especially for conversation archives and signal discovery.

13.4 Analytics Store

ClickHouse

- **Type:** Columnar analytics database
- **Why we use it:** The architecture explicitly defines ClickHouse as the analytics layer for revenue metrics, coaching metrics, and activity streams, and places it in Phase 2 for analytical workloads.
- **Primary use:** Fast analytical queries on large event and metric datasets.
- **Key use cases in development:**
 - dashboard metrics
 - coaching insights
 - revenue performance analysis
 - activity aggregation
- **Why it matters:** Analytical workloads should not overload the transactional database.

Analytics separation principle

- **Type:** Architecture rule
- **Why we use it:** The architecture separates transactional storage from search and analytics stores, which is important to keep the system scalable and predictable.
- **Primary use:** Put the right workload in the right store.
- **Key use cases in development:**
 - keep transactional DB healthy
 - avoid heavy reporting queries on PostgreSQL
 - support better dashboard performance
- **Why it matters:** Prevents one store from becoming a bottleneck for every workload.

13.5 Object / File Storage

Supabase Storage or equivalent object storage

- **Type:** Object/file storage
- **Why we use it:** The platform deals with recordings, files, transcripts, and large artifacts, so file storage must sit outside the primary relational database for operational sanity.
- **Primary use:** Store files and large binary artifacts.
- **Key use cases in development:**
 - uploaded files
 - meeting recordings
 - intermediate processing artifacts
 - export bundles
- **Why it matters:** Keeps large files out of PostgreSQL and supports cleaner storage workflows.

Audio retention and deletion policy

- **Type:** Storage governance rule
- **Why we use it:** The architecture explicitly states that the platform owns transcripts and metadata, not long-term raw audio, and that audio should be auto-deleted after a configurable retention period, with 7 days used as the default compliance and cost-control rule.
- **Primary use:** Control storage cost and privacy exposure.
- **Key use cases in development:**
 - transcript-first storage strategy
 - compliance-safe audio handling
 - automated cleanup workflows
- **Why it matters:** Raw audio is expensive and sensitive, so we should not keep it longer than needed.

13.6 Data Export / Warehouse Connectors

Data Cloud export layer

- **Type:** Data export architecture
- **Why we use it:** The architecture explicitly defines Data Cloud in M-03 as the customer data export mechanism and records a daily idempotent sync approach in ADR-009.
- **Primary use:** Export tenant-owned data to customer-owned analytics systems.
- **Key use cases in development:**
 - full tenant data export
 - warehouse sync jobs
 - customer analytics enablement
 - external BI readiness
- **Why it matters:** The architecture treats customer ownership of data as a core principle.

Snowflake connector

- **Type:** Warehouse export target
- **Why we use it:** Snowflake is explicitly listed as a supported Data Cloud destination in the architecture.
- **Primary use:** Export data to customer-owned Snowflake environments.
- **Key use cases in development:**
 - customer analytics pipelines
 - downstream reporting
 - enterprise data integration
- **Why it matters:** Important for enterprise reporting and data ownership.

BigQuery connector

- **Type:** Warehouse export target
- **Why we use it:** BigQuery is explicitly listed as a supported Data Cloud destination in the architecture.
- **Primary use:** Export data to customer-owned BigQuery environments.
- **Key use cases in development:**
 - analytics exports

- customer-owned reporting pipelines
- batch data sync
- **Why it matters:** Supports customer analytics ecosystems.

Databricks connector

- **Type:** Warehouse / lakehouse export target
- **Why we use it:** Databricks is explicitly listed as a Data Cloud target in the architecture.
- **Primary use:** Export platform data to enterprise lakehouse environments.
- **Key use cases in development:**
 - enterprise data science workflows
 - analytics pipelines
 - customer-controlled downstream processing
- **Why it matters:** Useful for larger customers with advanced data platforms.

Amazon S3 and Redshift connectors

- **Type:** Data export targets
- **Why we use them:** The architecture explicitly includes Amazon S3 and Amazon Redshift as outbound data destinations in the Data Cloud design.
- **Primary use:** Customer-owned export and storage paths.
- **Key use cases in development:**
 - raw or structured export delivery
 - customer data backups
 - downstream warehouse loading
- **Why it matters:** Supports flexible customer-owned data strategies.

Connector rule

- **Type:** Governance principle
- **Why we use it:** The architecture explicitly states that warehouse credentials are client-provided and that Relanto.ai does not own customer warehouse infrastructure or directly host customer analytics.
- **Primary use:** Protect data ownership boundaries.
- **Key use cases in development:**
 - secure export-only design
 - avoid direct customer warehouse operations
 - maintain platform boundary discipline
- **Why it matters:** It keeps data ownership clear and reduces compliance risk.

14. Infrastructure and DevOps

This section defines the infrastructure and DevOps stack approved for R-Revenue Intelligence. The architecture explicitly lists Docker, GitHub Actions, Railway, Sentry, Grafana, Better Stack, Cloudflare, and Doppler as the core infrastructure layer, with Railway used in Phase 1 and Phase 2 and AWS ECS/Fargate planned as the scale-up target in Phase 3.

Why this matters in development:

- it gives the team one standard way to run, ship, and monitor the system
- it avoids every engineer inventing a separate local or deployment setup
- it keeps the stack simple enough for freshers while still being production-minded
- it supports the approved modular monolith now and future service extraction later

14.1 Local Development

Local development must stay as close as possible to the real architecture while remaining simple enough for daily engineering work. The architecture explicitly states that all services run in containers and that local

development uses Docker Compose.

Docker Compose

- **Type:** Local orchestration tool
- **Why we use it:** The architecture explicitly states that local development uses Docker Compose so engineers can run the main services together in a predictable setup.
- **Primary use:** Run the local stack consistently.
- **Key use cases in development:**
 - start frontend, backend, Python AI services, PostgreSQL, and Redis together
 - reduce onboarding friction for new engineers
 - make local debugging closer to production behavior
 - avoid “works on my machine” issues
- **Why it matters:** It gives one simple command path for local startup and reduces environment mismatch.

Local parity principle

- **Type:** DevOps operating principle
- **Why we use it:** The architecture separates product services, AI services, queueing, and storage, so local development must reflect those boundaries instead of hiding them behind fake shortcuts.
- **Primary use:** Keep development behavior realistic.
- **Key use cases in development:**
 - test queue-based workflows locally
 - run service-to-service communication locally
 - validate environment variables and startup dependencies
- **Why it matters:** It helps catch integration problems early instead of discovering them only after deployment.

14.2 Containerization

Docker

- **Type:** Containerization platform
- **Why we use it:** The architecture explicitly says all services run as containers and lists Docker as the standard packaging mechanism for the frontend, NestJS monolith, transcription service, and AI services layer.
- **Primary use:** Package and run services consistently.
- **Key use cases in development and deployment:**
 - containerize Next.js app
 - containerize NestJS backend
 - containerize FastAPI transcription service
 - containerize FastAPI AI services layer
 - standardize environment setup across machines and environments
- **Why it matters:** It creates predictable runtime behavior and makes deployment much easier.

Service-specific container boundaries

- **Type:** Architecture-aligned container strategy
- **Why we use it:** The architecture clearly separates the NestJS monolith, the transcription service, and the AI services layer because they have different runtime and scaling profiles.
- **Primary use:** Isolate components by workload type.
- **Key use cases in development:**
 - keep Python AI dependencies separate from TypeScript product services
 - isolate heavy Whisper workloads from the main app runtime
 - scale AI-heavy services differently from business APIs
- **Why it matters:** Different workloads need different memory, scaling, and failure boundaries.

14.3 Hosting

Railway

- **Type:** Managed hosting platform
- **Why we use it:** The architecture explicitly defines Railway as the deployment platform for Phase 1 and Phase 2 because it offers fast setup, health checks, restarts, and simple managed container hosting.
- **Primary use:** Early-stage hosting for the application stack.
- **Key use cases in development and delivery:**
 - MVP deployment
 - staging environments
 - early production rollout
 - fast service deployment without heavy infra overhead
- **Why it matters:** The architecture chose Railway because Phase 1 delivery speed is critical and the team does not need full microservice-grade infra complexity yet.

AWS ECS / Fargate

- **Type:** Planned scale-up hosting platform
- **Why we use it:** The architecture explicitly states that Railway is the early hosting path and AWS ECS/Fargate is the Phase 3 upgrade target once scale triggers are met.
- **Primary use:** More scalable production hosting later.
- **Key use cases in development and future operations:**
 - handle higher tenant counts
 - support stronger operational control
 - scale extracted services independently
 - improve long-term production resilience
- **Why it matters:** It gives a clear migration path without overcomplicating the MVP.

Hosting migration trigger

- **Type:** Infrastructure decision rule
- **Why we use it:** The architecture explicitly defines a migration trigger from Railway to AWS ECS when tenant or request scale exceeds the current model.
- **Primary use:** Prevent premature infra complexity.
- **Key use cases in development:**
 - keep current infra simple
 - avoid early platform overengineering
 - prepare clean migration points
- **Why it matters:** Teams often waste time on enterprise infra too early; this rule helps avoid that.

14.4 CI/CD

GitHub Actions

- **Type:** CI/CD automation platform
- **Why we use it:** The architecture explicitly defines GitHub Actions as the CI/CD layer for type checks, tests, Zod validation, Prisma validation, RLS checks, module-boundary checks, and deployment-related quality gates.
- **Primary use:** Automate build and validation workflows.
- **Key use cases in development:**
 - TypeScript type checking
 - unit and integration tests
 - schema validation
 - boundary enforcement
 - pre-merge quality checks

- **Why it matters:** It keeps quality gates consistent and prevents broken code from reaching shared branches.

PR quality gates

- **Type:** CI policy
- **Why we use it:** The architecture explicitly lists required PR checks such as type checking, tests, Zod schema validation, Prisma validation, RLS enforcement, no cross-module imports, and Sentry sourcemap upload.
- **Primary use:** Enforce architecture rules automatically.
- **Key use cases in development:**
 - catch schema mistakes early
 - prevent module-coupling violations
 - ensure multi-tenant safety checks
 - validate production readiness before merge
- **Why it matters:** Good CI acts like a second reviewer and protects the codebase from accidental regressions.

Dependency vulnerability scanning

- **Type:** CI security hygiene policy
- **Why we use it:** As dependency count grows across frontend, backend, and AI services, vulnerability scanning must be automatic rather than ad hoc.
- **Primary use:** Detect known vulnerable dependencies before release.
- **Key use cases in development:**
 - run JavaScript dependency audit checks (`npm audit` or equivalent) in CI
 - run Python dependency audit checks (`pip-audit`) for AI services
 - block high/critical unresolved vulnerabilities unless exception is approved
 - track remediation ownership in the team that owns the dependency
- **Why it matters:** It closes a basic but critical security hygiene gap for a fast-moving multi-stack codebase.

14.5 Secrets Management

Doppler

- **Type:** Secrets management platform
- **Why we use it:** The architecture explicitly states that Doppler manages all environment variables and that no secrets should exist in repos or container images. It also lists API key rotation through Doppler for AI service integrations.
- **Primary use:** Centralized secret storage and delivery.
- **Key use cases in development and operations:**
 - OpenAI API keys
 - Whisper / AssemblyAI credentials
 - CRM OAuth secrets
 - database credentials
 - service configuration
- **Why it matters:** Secret sprawl is a major operational and security risk, and a central system reduces that risk.

Secrets discipline

- **Type:** Security rule
- **Why we use it:** The architecture directly says secrets must not live in repos or images and must be handled centrally.
- **Primary use:** Reduce leakage risk.
- **Key use cases in development:**

- safe local environment setup
- controlled rotation
- fewer accidental credential leaks
- **Why it matters:** This is a basic but critical DevOps hygiene rule.

14.7 Required API Keys and Credentials

The platform requires several external API keys and connection endpoints for its core functionality. These must be managed securely via Doppler.

Environment Variable	Service	Primary Use Case
GROQ_API_KEY	Groq Cloud	Powers high-performance AI models (LLaMA 3.3)
SUPABASE_URL	Supabase	Database and managed services endpoint
SUPABASE_KEY	Supabase	Authentication and data access credentials
OPENAI_API_KEY	OpenAI	Primary LLM inference and embeddings
ASSEMBLYAI_API_KEY	AssemblyAI	Fallback ASR and diarization
SENDGRID_API_KEY	SendGrid	Transactional email delivery
HUBSPOT_ACCESS_TOKEN	HubSpot	CRM integration and sync
SALESFORCE_CLIENT_ID	Salesforce	CRM integration and sync

14.6 CDN / WAF / Edge Services

Cloudflare

- **Type:** CDN, WAF, edge, TLS, and rate-limiting platform
- **Why we use it:** The architecture explicitly lists Cloudflare for CDN, DDoS protection, TLS termination, WAF, and rate limiting. It also specifically calls out Cloudflare rate limits as part of webhook protection for conferencing integrations.
- **Primary use:** Protect and accelerate external traffic.
- **Key use cases in development and production:**
 - TLS termination
 - edge protection
 - DDoS defense
 - webhook rate limiting
 - frontend delivery optimization
- **Why it matters:** The platform has externally exposed integration endpoints, especially webhook entry points, so edge protection is not optional.

Webhook protection use case

- **Type:** Security-specific use of edge services
- **Why we use it:** The architecture explicitly says conferencing webhooks are mission-critical entry points and pairs Cloudflare rate limiting with HMAC verification and queue prioritization to prevent abuse.
- **Primary use:** Protect ingestion entry points.
- **Key use cases in development:**
 - Zoom webhook protection
 - Teams webhook protection
 - Google Meet webhook protection
 - inbound call recording event protection
- **Why it matters:** A malicious or noisy webhook path can flood queues and create direct cost and uptime problems.

15. Observability and Security

This section defines how we track errors, logs, uptime, incidents, and operational safety across the platform. The architecture explicitly lists Sentry, Grafana, Better Stack, Cloudflare, Doppler, RLS, RBAC, and compliance enforcement as the core observability and security stack, and it treats these as platform-level requirements rather than optional add-ons.

Why this matters in development:

- production issues become visible faster
- debugging gets easier across frontend, backend, and AI services
- multi-tenant and integration-heavy behavior stays safer
- the team can scale operations without losing control

15.1 Error Tracking

Sentry

- **Type:** Error tracking platform
- **Why we use it:** The architecture explicitly states that Sentry is used for error tracking across Next.js, NestJS, and Python services, and that every exception should be logged there.
- **Primary use:** Capture application errors and trace failures.
- **Key use cases in development and production:**
 - frontend crash visibility
 - backend exception tracking
 - AI service failure monitoring
 - release regression detection
- **Why it matters:** It gives one central place to find and triage runtime failures.

Sentry sourcemaps and release visibility

- **Type:** Error-debugging support pattern
- **Why we use it:** The architecture includes Sentry sourcemap upload as part of CI, which improves production debugging quality.
- **Primary use:** Better debugging of minified frontend errors.
- **Key use cases in development:**
 - map production stack traces back to source
 - tie errors to releases
 - shorten debugging time
- **Why it matters:** Faster debugging means lower MTTR and fewer blind spots.

15.2 Logging

Better Stack

- **Type:** Log aggregation and uptime platform
- **Why we use it:** The architecture explicitly lists Better Stack for log aggregation, structured log search, and uptime monitoring.
- **Primary use:** Centralized logging and searchable operational data.
- **Key use cases in development and operations:**
 - inspect structured app logs
 - investigate queue failures
 - trace integration errors
 - search historical incidents
- **Why it matters:** Logs are essential for debugging problems that are not obvious from error trackers alone.

Structured logging discipline

- **Type:** Logging practice
- **Why we use it:** The architecture emphasizes monitored data flows, event-driven processing, and integration reliability, which all work better when logs are structured and searchable.
- **Primary use:** Make logs machine-usable and human-readable.
- **Key use cases in development:**
 - correlation across services
 - tenant-safe diagnostics
 - queue and event tracing
 - integration retry analysis
- **Why it matters:** Unstructured logs become noise very quickly in distributed systems.

15.3 Monitoring and Alerting

Grafana

- **Type:** Metrics and dashboard platform
- **Why we use it:** The architecture explicitly lists Grafana for dashboards covering queue depths, DB pool behavior, AI latency, and error rates, with future expansion toward distributed tracing.
- **Primary use:** Operational metrics dashboards.
- **Key use cases in development and production:**
 - queue health monitoring
 - latency tracking
 - DB and Redis visibility
 - AI pipeline performance dashboards
- **Why it matters:** Metrics help us catch degradation before users report it.

Better Stack uptime monitoring

- **Type:** Uptime and alerting support
- **Why we use it:** The architecture explicitly includes Better Stack for uptime monitoring in addition to log aggregation.
- **Primary use:** Availability checks and alerts.
- **Key use cases in development and production:**
 - service health checks
 - endpoint availability alerts
 - external-facing route monitoring
- **Why it matters:** It helps detect outage symptoms quickly.

Alerting on integration and webhook failures

- **Type:** Operational safety pattern
- **Why we use it:** The architecture explicitly states that external integrations must be monitored and specifically mentions Sentry alerts and rejection-rate monitoring for webhook protections.
- **Primary use:** Catch integration failures early.
- **Key use cases in development:**
 - webhook rejection spikes
 - AI failure bursts
 - CRM sync retries
 - queue backlog alerts
- **Why it matters:** Many platform features depend on external systems, so silent failures are dangerous.

15.4 Incident Management

Incident workflow maturity

- **Type:** Operational process category
- **Why we use it:** The architecture highlights failure modes, single points of failure, RTO/RPO expectations, and escalation-worthy platform risks, which means incident handling must be deliberate and not ad hoc.
- **Primary use:** Respond to production problems consistently.
- **Key use cases in development and operations:**
 - outage handling
 - degraded AI provider response
 - Redis failure response
 - webhook flood or rejection incidents
- **Why it matters:** A good incident process reduces panic and improves recovery speed.

PagerDuty or equivalent later

- **Type:** Planned incident management tool
- **Why we use it:** While not shown as a hard Phase 1 dependency, the architecture references stronger on-call and SRE-style maturity later, making a dedicated incident platform a reasonable scale-up tool.
- **Primary use:** On-call routing and escalation at scale.
- **Key use cases in future operations:**
 - on-call notifications
 - escalation policies
 - incident response coordination
- **Why it matters:** As the platform scales, manual alert routing becomes unreliable.

15.5 Security and Compliance Tools

Row-Level Security (RLS)

- **Type:** Database security control
- **Why we use it:** The architecture explicitly defines shared PostgreSQL with row-level security per tenant and requires `tenant_id`-based isolation on every table and write path.
- **Primary use:** Prevent cross-tenant data access.
- **Key use cases in development:**
 - tenant-safe queries
 - DB enforcement of isolation
 - multi-tenant protection beyond app logic
- **Why it matters:** This is one of the most important security controls in the whole platform.

RBAC

- **Type:** Application authorization model
- **Why we use it:** The architecture explicitly defines role-based access control across modules and user types such as AE, SDR, Sales Manager, VP Sales, CRO, and RevOps.
- **Primary use:** Restrict features by role.
- **Key use cases in development:**
 - admin-only settings
 - manager views
 - role-aware dashboards
 - controlled access to configuration and exports
- **Why it matters:** Revenue intelligence data is sensitive and role boundaries must be enforced consistently.

Supabase Auth + JWT

- **Type:** Authentication and token-based security stack
- **Why we use it:** The architecture explicitly defines Supabase Auth with JWT verification and guarded routes as the platform authentication pattern.

- **Primary use:** Secure user identity and protected access.
- **Key use cases in development:**
 - sign-in flows
 - protected APIs
 - user identity management
 - session-to-role mapping
- **Why it matters:** It gives us secure auth without building a custom auth system from scratch.

Configure Compliance Settings

- **Type:** Platform compliance feature
- **Why we use it:** The architecture explicitly lists Configure Compliance Settings as a deployed cross-cutting feature that enforces CRM opt-out preferences and GDPR/CCPA-style policies.
- **Primary use:** Enforce communication and data-governance rules.
- **Key use cases in development:**
 - opt-out enforcement
 - regional communication policies
 - safe automation controls
- **Why it matters:** Compliance rules must be part of platform behavior, not left to manual discipline.

HMAC webhook verification

- **Type:** Integration security control
- **Why we use it:** The architecture explicitly requires HMAC-SHA256 verification for conferencing webhooks and treats webhook security as mission-critical.
- **Primary use:** Validate inbound third-party webhook authenticity.
- **Key use cases in development:**
 - Zoom webhook verification
 - Teams webhook verification
 - Google Meet or telephony event verification
- **Why it matters:** Unverified webhook traffic can create cost spikes, junk ingestion, and service abuse.

Audit and retention controls

- **Type:** Governance and compliance control category
- **Why we use it:** The architecture explicitly enforces audio retention limits, review processes, and client-data ownership rules.
- **Primary use:** Reduce privacy and compliance risk.
- **Key use cases in development:**
 - audio auto-deletion
 - safe CRM field write-back
 - controlled data export behavior
- **Why it matters:** Compliance is built from many small enforced rules, not one big checkbox.

16. External APIs and Integrations

This section defines the external systems that R-Revenue Intelligence connects with. The architecture is very clear that these systems sit outside the platform boundary and that we do not own them. We read from them, write approved outputs back to them, or use them as communication channels, but we do not replace their core role.

Why this matters in development:

- integrations are a major source of platform value
- integrations are also a major source of failure risk
- every integration needs auth, retries, monitoring, and idempotency
- the team must know what we own and what we do not own

16.1 CRM Integrations

Salesforce

- **Type:** External CRM integration
- **Why we use it:** The architecture explicitly lists Salesforce as a bidirectional CRM integration where accounts, contacts, opportunities, stages, and field values flow in, while AI enrichment fields, summaries, next steps, and risk signals can flow back out.
- **Primary use:** CRM sync and AI enrichment.
- **Key use cases in development:**
 - read deal and contact context
 - map conversation data to revenue entities
 - write approved AI enrichment fields back
 - support account and deal intelligence workflows
- **Why it matters:** Salesforce is a common enterprise system of record, so this integration is core to platform usefulness.

HubSpot

- **Type:** External CRM integration
- **Why we use it:** The architecture explicitly lists HubSpot as a bidirectional CRM integration for contacts, companies, deals, pipeline stages, AI fields, and email activity.
- **Primary use:** CRM sync for SMB and growth-focused customers.
- **Key use cases in development:**
 - company and deal sync
 - AI enrichment write-back
 - contact context mapping
 - workflow support for engagement features
- **Why it matters:** It broadens platform fit beyond Salesforce-heavy customers.

Microsoft Dynamics 365

- **Type:** External CRM integration
- **Why we use it:** The architecture explicitly lists Dynamics 365 as a bidirectional CRM integration for accounts, contacts, opportunities, AI fields, and activity logs.
- **Primary use:** Enterprise CRM sync and enrichment.
- **Key use cases in development:**
 - account context sync
 - opportunity mapping
 - activity log write-back
 - support enterprise CRM environments
- **Why it matters:** Important for enterprise customers using Microsoft ecosystems.

CRM integration rule

- **Type:** Governance principle
- **Why we use it:** The architecture explicitly states that the CRM remains the system of record and that R-Revenue Intelligence should only read it and write approved AI enrichment fields, not take ownership of CRM core state.
- **Primary use:** Respect system boundaries.
- **Key use cases in development:**
 - read-only sync for core CRM entities
 - controlled write-back fields
 - avoid writing deal stage changes as if we owned the CRM
- **Why it matters:** This prevents trust and governance problems with customer systems.

16.2 Meeting / Call Integrations

Zoom

- **Type:** Meeting / call recording integration
- **Why we use it:** The architecture explicitly lists Zoom as an inbound integration for call recordings, metadata, and participant details through HMAC-protected webhooks.
- **Primary use:** Capture meeting and call data.
- **Key use cases in development:**
 - receive recording events
 - ingest metadata
 - trigger transcription pipelines
- **Why it matters:** It is one of the key entry points for the platform's capture stage.

Google Meet

- **Type:** Meeting integration
- **Why we use it:** The architecture explicitly lists Google Meet as an inbound source of call recordings, metadata, and participants in Phase 1.
- **Primary use:** Capture meeting data from Google ecosystem users.
- **Key use cases in development:**
 - meeting data ingestion
 - transcript pipeline triggers
 - participant context capture
- **Why it matters:** Important for customers using Google Workspace.

Microsoft Teams

- **Type:** Meeting integration
- **Why we use it:** The architecture explicitly lists Microsoft Teams as an inbound conferencing integration with HMAC-protected webhook handling in Phase 1.
- **Primary use:** Capture conversations from Microsoft collaboration environments.
- **Key use cases in development:**
 - recording ingestion
 - metadata intake
 - participant capture
- **Why it matters:** Important for enterprise customers in Microsoft ecosystems.

Dialers / telephony tools

- **Type:** Call recording integration category
- **Why we use it:** The architecture explicitly includes telephony and dialer systems as inbound sources of call recordings, metadata, and outcomes.
- **Primary use:** Bring customer call data into the capture pipeline.
- **Key use cases in development:**
 - ingest phone call recordings
 - trigger ASR workflows
 - attach outcomes and metadata
- **Why it matters:** The product is not a dialer, but it must still work with telephony-generated conversation data.

16.3 Email Integrations

Gmail

- **Type:** Email and calendar integration
- **Why we use it:** The architecture explicitly lists Gmail as a bidirectional integration where email content, metadata, and calendar events flow in, while AI-composed emails flow out using OAuth2.
- **Primary use:** Email capture and AI-assisted outbound email.
- **Key use cases in development:**

- ingest message context
- support AI email composer
- link calendar and email activity
- trigger engagement workflows
- **Why it matters:** Email is a major signal source in revenue workflows.

Outlook / Office 365

- **Type:** Email and calendar integration
- **Why we use it:** The architecture explicitly lists Outlook / Office 365 as a bidirectional email and calendar integration, also based on OAuth2 delegation.
- **Primary use:** Microsoft-side email capture and sending support.
- **Key use cases in development:**
 - ingest Outlook emails
 - support AI-composed email delivery
 - sync calendar context
 - enrich engagement workflows
- **Why it matters:** Important for enterprise Microsoft environments.

Email boundary rule

- **Type:** Platform boundary principle
- **Why we use it:** The architecture explicitly states that R-Revenue Intelligence does not own email infrastructure and must use Gmail or Outlook APIs only, with no Relanto.ai MX-based sending model.
- **Primary use:** Keep email infrastructure external.
- **Key use cases in development:**
 - delegated sending only
 - no custom SMTP ownership
 - platform focuses on composition, not inbox infrastructure
- **Why it matters:** This keeps scope and deliverability risk under control.

16.4 Messaging / Collaboration Integrations

Slack

- **Type:** Messaging and collaboration integration
- **Why we use it:** The architecture explicitly lists Slack as an outbound integration for alerts, notifications, and to-do reminders, planned for Phase 3.
- **Primary use:** Notify users outside the app.
- **Key use cases in development:**
 - real-time alerts
 - competitor mention notifications
 - reminder delivery
 - workflow visibility
- **Why it matters:** Collaboration tools help surface signals where teams already work.

Collaboration integration rule

- **Type:** Workflow extension principle
- **Why we use it:** The architecture treats collaboration tools as outbound channels, not as core systems of record.
- **Primary use:** Extend product usefulness beyond the app UI.
- **Key use cases in development:**
 - send notifications
 - deliver reminders
 - surface action items
- **Why it matters:** It keeps the platform workflow-friendly without expanding its scope too much.

16.5 Billing Integrations

Billing provider placeholder

- **Type:** Business operations integration category
- **Why we use it:** The current architecture excerpt does not define a formal billing provider such as Stripe, so billing integrations should be treated as not-yet-approved and require explicit review before inclusion.
- **Primary use:** Future subscription and payment workflows.
- **Key use cases in future development:**
 - subscription management
 - invoicing support
 - plan and seat billing
- **Why it matters:** Billing is important, but it should not be added casually outside the approved architecture.

Recommendation for billing tools

- **Type:** Governance note
- **Why we use it:** Because billing is not currently part of the approved architecture stack, any future billing tool should go through the same tool review process as other external systems.
- **Primary use:** Keep architecture discipline intact.
- **Key use cases in development:**
 - avoid unreviewed SaaS adoption
 - align billing model with modular pricing strategy
- **Why it matters:** Business tooling can create deep long-term dependencies too.

16.6 Other Third-Party Services

OpenAI API

- **Type:** External AI service
- **Why we use it:** The architecture explicitly lists OpenAI API as a core external dependency for summaries, Q&A, email drafts, and RAG workflows, called only from Python AI services.
- **Primary use:** Cloud LLM inference.
- **Key use cases in development:**
 - summary generation
 - extraction
 - question answering
 - AI writing workflows
- **Why it matters:** It is one of the highest-value external services in the stack.

Whisper / AssemblyAI

- **Type:** External AI / speech services
- **Why we use them:** The architecture explicitly lists Whisper as the primary ASR dependency and AssemblyAI as the fallback and diarization-support path.
- **Primary use:** Speech-to-text and backup audio intelligence.
- **Key use cases in development:**
 - transcript generation
 - fallback transcription
 - speaker-aware transcript workflows
- **Why it matters:** Transcription is the first step for much of the product value chain.

LinkedIn Sales Navigator

- **Type:** Sales intelligence integration

- **Why we use it:** The architecture explicitly lists LinkedIn Sales Navigator as an inbound source of prospect profiles and connection activity in Phase 2.
- **Primary use:** Prospect enrichment and sales context.
- **Key use cases in development:**
 - prospect context capture
 - engagement intelligence
 - sales research workflows
- **Why it matters:** Helps connect external prospect context to revenue workflows.

Warehouse targets

- **Type:** Customer-owned export destinations
- **Why we use them:** The architecture explicitly lists Snowflake, BigQuery, Databricks, Amazon S3, and Redshift as outbound data-export targets through Data Cloud.
- **Primary use:** Deliver full customer-owned data exports.
- **Key use cases in development:**
 - analytics export
 - warehouse sync
 - customer-owned reporting
 - BI enablement
- **Why it matters:** Data ownership is a core architecture principle.

17. Testing Stack

This section defines the approved testing stack for R-Revenue Intelligence. The architecture makes testing a platform-level requirement, not a nice-to-have, and repeatedly emphasizes integration correctness, end-to-end event flow validation, RLS enforcement, schema validation, and safe AI handling. Because the platform is modular, event-driven, multi-tenant, and AI-assisted, we need different test layers for different risks.

Why this matters in development:

- unit tests catch logic bugs early
- integration tests verify real module boundaries
- end-to-end tests validate user workflows and data flows
- load tests protect us from queue, AI, and webhook bottlenecks
- AI evaluation tests help us improve prompts and outputs without breaking trust

17.1 Unit Testing

Unit tests validate small pieces of logic in isolation. They are the fastest test layer and are best for business rules, utility functions, validation logic, state transformations, formatting helpers, guards, and service methods.

Jest

- **Type:** TypeScript unit testing framework
- **Why we use it:** The architecture explicitly includes Jest in PR quality gates and requires unit plus integration coverage before merge.
- **Primary use:** Unit testing for frontend and backend TypeScript code.
- **Key use cases in development:**
 - NestJS service logic
 - DTO and transformer testing
 - utility and helper functions
 - RBAC guard behavior
 - frontend component logic and hooks

- **Why it matters:** It is the standard testing layer for TypeScript-heavy product code and works well with both frontend and backend workflows.

pytest

- **Type:** Python unit testing framework
- **Why we use it:** The architecture separates Python AI services from TypeScript product services, so Python-side logic needs its own reliable unit test layer.
- **Primary use:** Unit testing for AI and ML service code.
- **Key use cases in development:**
 - FastAPI helper functions
 - NLP pipeline utilities
 - embedding logic
 - prompt formatter functions
 - forecast preprocessing logic
- **Why it matters:** AI services should be tested as real software, not treated like magic.

Unit testing rule

- Use unit tests for isolated logic only.
- Do not use unit tests as a replacement for integration or event-flow tests.
- Mock expensive or external dependencies in unit tests, especially LLM calls, speech APIs, and external integrations.

17.2 Integration Testing

Integration tests verify that real components work together correctly. In this project, integration testing is especially important because the architecture depends on service boundaries, queues, schemas, OAuth-style integrations, multi-tenant data protection, and event-driven workflows.

Supertest

- **Type:** API integration testing library
- **Why we use it:** It is well suited for testing real NestJS HTTP endpoints with auth, validation, guards, and response contracts in place.
- **Primary use:** Backend API integration tests.
- **Key use cases in development:**
 - authenticated route tests
 - payload validation tests
 - RBAC route access tests
 - error response behavior
 - contract verification
- **Why it matters:** It tests the real HTTP layer instead of only internal functions.

Testcontainers

- **Type:** Real dependency test environment tool
- **Why we use it:** The architecture depends on PostgreSQL, Redis, and queue-based processing, so realistic infrastructure-backed tests are much more valuable than pure mocks.
- **Primary use:** Integration testing with real infra dependencies.
- **Key use cases in development:**
 - PostgreSQL-backed service tests
 - Redis and BullMQ integration tests
 - startup and migration checks
 - RLS verification against real DB behavior
- **Why it matters:** It increases confidence that the application will behave correctly outside the developer laptop.

Prisma + Zod + RLS validation scripts

- **Type:** Schema and governance integration checks
- **Why we use them:** The architecture explicitly requires Prisma schema validation, Zod schema validation, and RLS enforcement checks on every PR.
- **Primary use:** Verify data contracts and tenant-safety controls.
- **Key use cases in development:**
 - schema correctness
 - no invalid request handling paths
 - all tables protected by RLS
 - tenant isolation enforcement
- **Why it matters:** These checks protect platform correctness and customer isolation, not just developer convenience.

Integration testing rule

- Use integration tests whenever the logic crosses:
 - HTTP boundaries
 - DB boundaries
 - queue boundaries
 - auth boundaries
 - event boundaries
- If the feature depends on another module, test the public contract, not internal private functions.

17.3 End-to-End Testing

End-to-end tests validate real product workflows across multiple layers, such as frontend, backend, auth, queues, and storage. These are especially important for workflows like login, transcription, CRM setup, dashboard loading, and key user journeys.

Playwright

- **Type:** Browser end-to-end testing framework
- **Why we use it:** Playwright is the best fit for realistic UI flows in a Next.js product with authenticated screens, tables, forms, filters, and module-driven dashboards.
- **Primary use:** Full browser-based workflow testing.
- **Key use cases in development:**
 - login and protected route checks
 - connector setup flows
 - transcript search UI
 - dashboard and reporting flows
 - regression testing on major user journeys
- **Why it matters:** It validates the app the way users actually use it.

Event-flow verification tests

- **Type:** Cross-module E2E system test pattern
- **Why we use it:** The architecture explicitly defines event-flow verification as a deployment rule and gives concrete examples such as `call.transcription.completed -> revenue_graph.entity.linked`.
- **Primary use:** Validate upstream-to-downstream module flow.
- **Key use cases in development:**
 - webhook to transcription to storage
 - transcription completion to downstream AI processing
 - tracker creation to summary refresh
 - queue-based retries and idempotency
- **Why it matters:** In this platform, successful E2E behavior depends heavily on events, not just screens.

E2E testing rule

- Use Playwright for UI journeys.
- Use event-flow system tests for async backend journeys.
- Never mark a module as production-ready only from unit test success; upstream and downstream flow must also work with realistic data.

17.4 Load and Performance Testing

Load and performance testing is important because the architecture already identifies scale-sensitive areas such as webhook ingestion, transcription throughput, Redis queue health, AI latency, PostgreSQL load, and future scaling triggers.

k6

- **Type:** Load and performance testing tool
- **Why we use it:** k6 is a practical way to simulate API load, concurrency, and response behavior for backend and integration-heavy workflows.
- **Primary use:** Performance and throughput validation.
- **Key use cases in development:**
 - API load testing
 - auth endpoint stress testing
 - webhook burst simulation
 - queue-enqueue throughput testing
 - p95 and p99 latency checks
- **Why it matters:** It helps us find bottlenecks before real users or customer sync jobs do.

Queue and AI pipeline performance tests

- **Type:** Async performance validation
- **Why we use them:** The architecture explicitly identifies Redis and BullMQ as critical path dependencies and documents scaling settings for transcription and AI services.
- **Primary use:** Validate async system stability under pressure.
- **Key use cases in development:**
 - queue depth growth behavior
 - retry storms
 - webhook spike handling
 - Whisper / AI service throughput checks
- **Why it matters:** A fast API is not enough if async jobs silently back up in the background.

Performance testing rule

- Test both sync and async paths.
- Measure API latency, queue wait time, job completion time, and failure rate.
- Include realistic payload sizes, especially for transcripts and audio-triggered workflows.

17.5 Test Data / Mocking / Containers

Good testing depends on good test data and realistic dependency handling. Because this platform works with calls, transcripts, CRM records, and AI outputs, we need both deterministic mocks and realistic test fixtures.

Mocking external services

- **Type:** Test strategy
- **Why we use it:** External dependencies such as OpenAI, Whisper, AssemblyAI, Salesforce, Gmail, and Slack should not be called directly in most automated test runs.

- **Primary use:** Keep tests deterministic and affordable.
- **Key use cases in development:**
 - mock LLM responses
 - mock transcription responses
 - mock CRM sync outcomes
 - simulate retries and error codes
- **Why it matters:** Tests should fail because our code is wrong, not because a third-party API had a bad day.

Test fixtures and golden samples

- **Type:** Reusable test data assets
- **Why we use them:** The architecture uses structured transcripts, CRM-linked entities, and AI-generated JSON outputs, so stable example payloads are very useful.
- **Primary use:** Reuse known-good examples across tests.
- **Key use cases in development:**
 - sample transcripts
 - CRM payload fixtures
 - event payload fixtures
 - expected AI JSON outputs
- **Why it matters:** Stable fixtures make regression testing much easier.

Containers for realistic test environments

- **Type:** Infra-backed testing pattern
- **Why we use them:** The architecture is container-first and local development already uses Docker Compose, so realistic container-backed testing is a natural fit.
- **Primary use:** Reproduce production-like dependencies.
- **Key use cases in development:**
 - PostgreSQL integration tests
 - Redis queue tests
 - migration validation
 - service startup tests
- **Why it matters:** It reduces surprises between local and deployed environments.

17.6 AI Evaluation and Prompt Testing

AI evaluation is a separate testing layer because AI quality is not fully covered by normal unit tests. The architecture explicitly requires structured JSON outputs, confidence scores, flagged review paths, and safe fallback handling, which means prompt quality and response quality need dedicated evaluation patterns.

Golden datasets

- **Type:** AI evaluation approach
- **Why we use it:** Prompt changes and model changes should be tested against known examples instead of relying only on intuition.
- **Primary use:** Detect AI quality regression.
- **Key use cases in development:**
 - compare summary quality
 - compare extraction quality
 - validate tracker detection
 - measure consistency after prompt updates
- **Why it matters:** It makes prompt work more engineering-driven and less guess-based.

Prompt versioning

- **Type:** Prompt management practice

- **Why we use it:** The architecture treats AI endpoints as versioned contracts and structured services, so prompts should also be treated as controlled artifacts.
- **Primary use:** Track prompt changes safely.
- **Key use cases in development:**
 - compare old vs new prompt behavior
 - rollback bad prompt changes
 - record why output changed
- **Why it matters:** Prompt drift without control creates silent regressions.

Confidence-score gating

- **Type:** AI safety mechanism
- **Why we use it:** The architecture explicitly requires AI services to return `confidence_score` and route low-confidence outputs to review instead of direct automation.
- **Primary use:** Safer AI deployment.
- **Key use cases in development:**
 - hold weak outputs for review
 - compare confidence over model versions
 - prevent incorrect CRM writes
- **Why it matters:** It adds a practical safety net around AI features.

Human review and prompt review loops

- **Type:** Human-in-the-loop evaluation process
- **Why we use it:** The architecture explicitly supports flagged review flows and avoids blind automation for uncertain AI outputs.
- **Primary use:** Improve quality with reviewer feedback.
- **Key use cases in development:**
 - review low-confidence summaries
 - inspect extraction mistakes
 - tune prompts with real examples
- **Why it matters:** Human review is often the fastest path to trustworthy AI improvement.

18. Tool Approval Status

This table tracks whether a tool is approved, already in use, planned for later, optional for experimentation, or explicitly rejected. The architecture makes it clear that technology choices are governed decisions, not personal preferences, and that new shared tools should not enter the codebase without review.

Use this table with the following meaning:

- **Approved:** Tool is part of the approved architecture or standard stack.
- **In Use:** Tool is already being used in active implementation or deployed scope.
- **Planned:** Tool is approved or expected for a later phase.
- **Optional:** Tool is allowed for experiments or local use but is not required.
- **Rejected:** Tool should not be introduced.
- **Decision Owner:** Usually Tech Lead, AI Lead, Frontend Lead, Backend Lead, DevOps Lead, or Security Owner.

Copy-paste table

Tool	Approved	In Use	Planned	Optional	Rejected	Decision Owner	Main Use Case	Notes
Next.js 15	Yes	Yes	No	No	No	Tech Lead /	Frontend framework	Approved in architecture

Tool	Approved	In Use	Planned	Optional	Rejected	Decision Owner	Main Use Case	Notes
						Frontend Lead		
Tailwind CSS	Yes	Yes	No	No	No	Frontend Lead	Frontend styling	Approved in architecture
shadcn/ui	Yes	Yes	No	No	No	Frontend Lead	Reusable UI components	Approved in architecture
TanStack Query	Yes	Yes	No	No	No	Frontend Lead	Server-state management	Approved in architecture
Zustand	Yes	Yes	No	No	No	Frontend Lead	Client-state management	Approved in architecture
React Hook Form	Yes	Yes	No	No	No	Frontend Lead	Form handling	Frontend form standard
Zod	Yes	Yes	No	No	No	Backend Lead	Validation	Approved in architecture
						Tech Lead / Backend Lead		
NestJS v11	Yes	Yes	No	No	No	Tech Lead / Backend Lead	Backend framework	Approved in architecture
Prisma	Yes	Yes	No	No	No	Backend Lead	ORM and migrations	Approved in architecture
						Tech Lead / Backend Lead		
Supabase Auth	Yes	Yes	No	No	No	Tech Lead / Backend Lead	Authentication	Approved in architecture
BullMQ	Yes	Yes	No	No	No	Backend Lead	Queue and event bus	Approved in architecture
						Backend Lead / DevOps Lead		
Redis	Yes	Yes	No	No	No	Backend Lead / DevOps Lead	Queue backing and cache	Critical path dependency
FastAPI	Yes	Yes	No	No	No	AI Lead	Python AI services	Approved in architecture
LiteLLM	Yes	Yes	No	No	No	AI Lead	LLM routing	Approved in architecture
						AI Lead / Tech Lead		
OpenAI API	Yes	Yes	No	No	No	AI Lead / Tech Lead	Primary LLM inference	Core external dependency
Whisper	Yes	Yes	No	No	No	AI Lead	Primary ASR	Critical path
AssemblyAI	Yes	Yes	No	No	No	AI Lead	Backup ASR / diarization	Approved fallback
LangGraph	Yes	No	Yes	No	No	AI Lead	Agent and workflow orchestration	Planned AI orchestration layer
spaCy	Yes	No	Yes	No	No	AI Lead	NLP preprocessing	Approved in AI stack
pgvector	Yes	No	Yes	No	No	Data Lead / AI	Embeddings and semantic	Approved vector strategy

Tool	Approved	In Use	Planned	Optional	Rejected	Decision Owner	Main Use Case	Notes
						Lead	search	
Meilisearch	Yes	No	Yes	No	No	Backend Lead / Search Owner	Search indexing	Planned search engine
ClickHouse	Yes	No	Yes	No	No	Data Lead	Analytics store	Planned analytics layer
Docker	Yes	Yes	No	No	No	DevOps Lead	Containerization	Approved in architecture
Docker Compose	Yes	Yes	No	No	No	DevOps Lead	Local development	Local standard
Railway	Yes	Yes	No	No	No	DevOps Lead / Tech Lead	Phase 1–2 hosting	Planned upgrade later
AWS ECS / Fargate	Yes	No	Yes	No	No	Tech Lead / DevOps Lead	Phase 3 hosting	Upgrade target
GitHub Actions	Yes	Yes	No	No	No	DevOps Lead	CI/CD	Approved in architecture
Doppler	Yes	Yes	No	No	No	DevOps Lead / Security Owner	Secrets management	Approved in architecture
Cloudflare	Yes	Yes	No	No	No	DevOps Lead / Security Owner	CDN / WAF / rate limiting	Approved in architecture
Sentry	Yes	Yes	No	No	No	DevOps Lead	Error tracking	Approved in architecture
Better Stack	Yes	Yes	No	No	No	DevOps Lead	Logs and uptime	Approved in architecture
Grafana	Yes	Yes	No	No	No	DevOps Lead	Metrics dashboards	Approved in architecture
Playwright	Yes	Yes	No	No	No	QA Lead / Frontend Lead	E2E testing	Recommended test stack
Jest	Yes	Yes	No	No	No	Backend Lead / Frontend Lead	Unit testing	Required in CI
pytest	Yes	Yes	No	No	No	AI Lead	Python tests	Python AI service testing
k6	Yes	No	Yes	No	No	QA Lead / DevOps Lead	Load testing	Recommended performance tool

Tool	Approved	In Use	Planned	Optional	Rejected	Decision Owner	Main Use Case	Notes
Testcontainers	Yes	No	Yes	No	No	Backend Lead / QA Lead	Infra-backed integration tests	Recommended for realistic test envs
Ollama	No	No	No	Yes	No	AI Lead	Local LLM experiments	Development / offline only [web:26]
LM Studio	No	No	No	Yes	No	AI Lead	Local AI and document chat	Experimentation / onboarding [page:1]
Google AI Studio	No	No	No	Yes	No	AI Lead	Fast prompt prototyping	Evaluation only [web:22]
Groq	No	No	No	Yes	No	AI Lead	Free API benchmarking	Prototype and latency testing [web:20]
Hugging Face hosted inference	No	No	No	Yes	No	AI Lead	Model experiments	Prototype and evaluation [page:3]
openai npm package in TypeScript services	No	No	No	No	Yes	Tech Lead	Direct LLM calls from product services	Explicitly forbidden by architecture
LangChain in TypeScript product services	No	No	No	No	Yes	Tech Lead	AI logic inside product layer	Explicitly forbidden by architecture
direct cross-module DB access	No	No	No	No	Yes	Tech Lead	Shortcut data sharing	Violates module ownership rules
Raw SMTP / custom email server	No	No	No	No	Yes	Tech Lead / Security Owner	Direct email infrastructure ownership	Explicitly outside platform boundary
LangChain	Yes	Yes	No	No	No	AI Lead	AI Orchestration	Approved for AI Layer
Axios	Yes	Yes	No	No	No	Frontend Lead / Backend Lead	HTTP Client	Standard communication tool
Socket.io	Yes	Yes	No	No	No	Backend Lead	Real-time comms	Real-time standard
Framer Motion	Yes	Yes	No	No	No	Frontend Lead	UX Animations	Approved for premium UI
CatBoost / XGBoost	Yes	Yes	No	No	No	AI Lead	Tabular ML	Approved for forecasting
CrewAI	Yes	No	Yes	No	No	AI Lead	Agentic workflows	Planned for autonomous agents

19. Payment Planning

This table helps the team plan which tools are already likely to cost money, which ones can stay on free tiers for now, and what should trigger upgrades later. The architecture already defines clear phase changes such as Railway to AWS ECS, Redis to HA, and scale-based upgrade triggers, so payment planning should be tied to those real technical milestones rather than vague guesses.

Use this table with the following meaning:

- **Current Plan:** Free, Trial, Starter, Production, or Not Yet Used
- **Expected Upgrade Trigger:** The specific event that should justify spending more
- **Estimated Future Cost:** Keep this rough unless procurement has real pricing
- **Priority:** Use High, Medium, or Low
- **Notes:** Include whether the upgrade is driven by scale, security, uptime, AI quality, or compliance

Copy-paste table

Tool / Service	Current Plan	Expected Upgrade Trigger	Estimated Future Cost	Main Use Case	Priority	Notes
OpenAI API	Production usage-based	When AI features expand beyond current Phase 1 scope or token usage rises sharply	High, usage-based	Summaries, Q&A, email drafts, structured AI outputs	High	Core AI dependency; architecture says this is primary LLM path
Whisper	Production usage-based or self-hosted cost	Higher audio volume, lower latency needs, or more private deployment requirements	Medium to High	Speech-to-text transcription	High	Critical revenue path
AssemblyAI	Production fallback usage	Increased fallback traffic or diarization-heavy usage	Medium	Backup ASR and diarization	Medium	Fallback only, so cost depends on failure and quality needs
Railway	Starter / managed hosting	More than 500 tenants or major traffic growth	Medium to High	Phase 1–2 hosting	High	Architecture defines AWS ECS as upgrade path
AWS ECS / Fargate	Not yet used	When Railway scale trigger is reached	High	Phase 3 hosting and service isolation	High	Upgrade only when justified by scale

Tool / Service	Current Plan	Expected Upgrade Trigger	Estimated Future Cost	Main Use Case	Priority	Notes
Supabase PostgreSQL	Managed early-stage plan	Higher DB load, enterprise requirements, or infra control needs	Medium to High	Primary relational database	High	Early-phase managed DB path
Redis / Upstash HA	Starter / single-node now	Any Redis downtime incident or stronger HA need	Medium	Queue backing and cache	High	Architecture explicitly says upgrade immediately on downtime incident
Cloudflare	Basic / starter	Increased traffic, stronger WAF needs, or advanced rate-limiting requirements	Low to Medium	CDN, WAF, TLS, rate limiting	High	Important even when cost is modest
Sentry	Free tier or starter	Higher event volume, release complexity, or team debugging needs	Medium	Error tracking	Medium	Useful to upgrade once real production noise grows
Better Stack	Free tier or starter	Higher log volume, longer retention needs, or more uptime checks	Medium	Logs and uptime monitoring	Medium	Upgrade with production maturity
Grafana stack	Starter / low-cost setup	More metrics, tracing, and multi-service visibility needs	Medium	Metrics dashboards	Medium	More valuable as services and traffic grow
Doppler	Starter / team plan	More environments, stronger secret governance, or larger engineering team	Low to Medium	Secrets management	Medium	Small cost, high value for hygiene
GitHub Actions	Included / usage-based	Higher CI minutes, more test matrix builds, or more	Low to Medium	CI/CD automation	Medium	Usually grows naturally with team and test coverage

Tool / Service	Current Plan	Expected Upgrade Trigger	Estimated Future Cost	Main Use Case	Priority	Notes
		preview pipelines				
ClickHouse	Not yet used	Analytics volume grows and dashboard workloads need separate scaling	Medium to High	Analytical storage	Medium	Planned Phase 2 analytics layer
Meilisearch	Not yet used	Search experience becomes central and transcript corpus grows	Medium	Product search	Medium	Planned search engine
pgvector	Included in PostgreSQL now	Any tenant approaches large vector limits or search performance degrades	Low to Medium initially	Embeddings and semantic search	Medium	Architecture warns about larger-scale vector triggers
Dedicated vector store later	Not yet used	Any tenant exceeds around 800k vectors or retrieval performance becomes a problem	Medium to High	Large-scale vector retrieval	Low initially	Future upgrade path implied by architecture trigger notes
Playwright CI runs	Basic CI usage	More browser matrices, more E2E coverage, or more PR concurrency	Low to Medium	E2E regression testing	Medium	Cost is usually CI time rather than direct licensing
k6 / performance environments	Not yet used	Before major launch, enterprise onboarding, or infra migration	Low to Medium	Load testing	Medium	Use before scale surprises appear
Slack integration	Not yet used	When Phase 3 notifications and reminders are enabled	Low to Medium	Alerts and reminders	Low	Outbound integration planned later
Snowflake / BigQuery /	Not yet used	When Data Cloud is	Medium	Customer-owned data	Medium	Customer credentials

Tool / Service	Current Plan	Expected Upgrade Trigger	Estimated Future Cost	Main Use Case	Priority	Notes
warehouse export ops		activated for customers		export		owned by client; our cost mainly operational
Ollama	Free local	Upgrade only if moving to managed local-serving infra or dedicated hardware	Low	Offline model experimentation	Low	Great for development; not a major SaaS cost [web:26]
LM Studio	Free local	Usually no upgrade needed unless enterprise tooling policy requires alternatives	Low	Local AI testing and demos	Low	Experimentation and local enablement [page:1]

Payment planning rule

Do not upgrade a tool just because the free tier ends. Upgrade only when one of these is true:

- the tool blocks delivery speed
- the tool blocks reliability
- the tool blocks security or compliance
- the tool blocks scale
- the paid plan clearly improves product quality enough to justify the spend

Payment planning ownership

For each paid or potentially paid tool, assign a clear owner:

- **Tech Lead** for stack-level approval
- **DevOps Lead** for infra and observability cost
- **AI Lead** for model and inference cost
- **Backend / Data Lead** for DB, search, and analytics cost
- **Product / Founder** for business-priority decisions when trade-offs are needed

That ownership model helps prevent surprise renewals, duplicate tooling, and untracked vendor sprawl.

20. Risks and Notes

This section captures the main technical and operational risks in the current stack. These are not theoretical risks; most of them come directly from architecture choices such as external AI dependency, modular extraction later, shared PostgreSQL with RLS, queue-based processing, and Phase 1 to Phase 3 migration plans.

Vendor lock-in risks

- **OpenAI dependency risk:** OpenAI is the primary LLM provider, so core AI workflows such as summaries, Q&A, and generation are sensitive to provider pricing, API changes, model deprecations, and rate limits. The architecture reduces this risk by routing through LiteLLM and keeping fallback paths available.
- **Railway dependency risk:** Railway is the approved hosting platform for Phase 1 and Phase 2, which is good for speed but creates a platform dependency until AWS ECS migration is completed.
- **Supabase dependency risk:** Supabase is used for PostgreSQL and authentication in the early architecture, so both data and auth have managed-service dependency risk. The document already identifies Supabase Auth and PostgreSQL as critical external dependencies.
- **Cloud provider migration complexity:** Even when abstractions exist, moving from managed services to self-managed or AWS-native infrastructure later will still require testing, operational work, and migration planning.

API cost risks

- **LLM cost concentration:** The architecture explicitly indicates that OpenAI usage is a major AI budget driver, with much of product value depending on transcription and downstream AI processing.
- **Speech processing cost risk:** Whisper and AssemblyAI usage can grow quickly when call volume increases because raw audio ingestion is a high-volume path in the system.
- **Retry amplification risk:** Because the system is event-driven and queue-based, bad retry policies or duplicate event handling can multiply AI and transcription cost if idempotency is not enforced correctly.
- **Silent cost growth risk:** AI systems can generate cost growth before revenue growth if prompt size, transcript size, or background AI jobs increase faster than customer conversion.

Free-tier limitations

- **Hosting free-tier limits:** Free or starter plans are fine for early development, but production traffic, logs, queues, and metrics can exceed free-tier limits very quickly. The architecture already assumes eventual upgrades for Redis HA, AWS hosting, and higher operational maturity.
- **Monitoring retention limits:** Free tiers for error tracking, logs, and observability often have poor retention, lower event quotas, or limited team visibility.
- **AI evaluation tooling limits:** Free online AI tools are useful for prototyping, but they are not reliable enough to be treated as stable product infrastructure. [web:22][web:20]
- **Search and analytics growth limits:** Search, embeddings, and analytics can start small on shared systems, but larger transcript and event volumes will eventually force dedicated scaling decisions such as ClickHouse maturity or vector-search optimization.

Privacy risks for free online tools

- **Sensitive data exposure risk:** The architecture clearly treats customer data ownership, tenant isolation, and consent as non-negotiable, so free online tools must never be used casually with customer transcripts, CRM data, or sensitive business context.
- **Training and retention uncertainty:** Free online tools may retain prompts, use data for product improvement, or change their data policy later, which creates serious privacy and compliance risk for business data. [web:22][web:20]
- **Workspace leakage risk:** If team members paste customer data into public chat tools during testing, that becomes a governance problem even if the product architecture itself is secure.
- **Safer rule:** Free online tools should be limited to synthetic data, sanitized samples, or public information unless explicitly approved.

Migration risks

- **Railway to AWS migration risk:** The architecture explicitly plans Railway in Phase 1–2 and AWS ECS in Phase 3, which means deployment, networking, scaling, secrets, observability, and incident workflows will all need migration effort later.

- **Module extraction risk:** The architecture uses a modular monolith now and plans service extraction later, especially for M-03, M-05, and M-07. Even with clean boundaries, extraction still carries deployment, latency, and ownership risk.
- **Search and analytics migration risk:** Moving from early simpler setups to dedicated search or analytics layers can introduce backfill, schema sync, and consistency risks.
- **Auth and infra migration risk:** The architecture already notes fallback or future alternatives such as Auth0 and stronger Redis HA, which means identity and infra migration paths must remain documented and tested.

Security considerations

- **Webhook security is critical:** The architecture explicitly calls conferencing webhooks mission-critical and requires HMAC-SHA256 verification, idempotency, Cloudflare rate limiting, queue prioritization, and alerting.
- **Tenant isolation must never rely on one layer only:** The architecture enforces defense in depth through RLS, Prisma middleware, JWT guards, tenant interceptors, and RBAC.
- **Secrets leakage risk:** API keys and service credentials must stay in Doppler or approved secret storage and must never be hardcoded in repos, images, or local files committed to source control.
- **TypeScript AI-call violation risk:** The architecture explicitly forbids direct AI calls from TypeScript product services, because it increases blast radius and breaks the approved separation of concerns.
- **Retention and deletion risk:** Raw audio is sensitive and expensive, so the document explicitly requires configurable auto-deletion, with 7 days used as the default retention rule.

General notes

- **Current reality:** Phase 1 is strong for capture and transcription, but many downstream modules still depend on M-03 and later stages that are not fully deployed yet.
- **Business promise risk:** Because the product is sold as modular, any architecture shortcut that weakens module independence directly threatens pricing and packaging strategy.
- **Freshers rule:** If a shortcut creates tighter coupling, bypasses events, bypasses tenant rules, or mixes AI logic into product code, it is probably the wrong decision for this platform.

21. Libraries That Will Be Used

This section lists the major libraries, frameworks, and core tools used across the platform, along with their main use cases. The selections below are based on the approved architecture, documented external dependencies, and the stack choices already established in the earlier sections.

Frontend libraries

Library / Tool	Layer	Use Case
Next.js 15	Frontend	Main web application framework, routing, SSR, layouts
TypeScript	Frontend	Type safety for components, API usage, and state
Tailwind CSS	Frontend	Utility-first styling and design consistency
shadcn/ui	Frontend	Reusable accessible UI components
Radix UI	Frontend	Accessible UI primitives for dialogs, popovers, menus
TanStack Query	Frontend	Server-state fetching, caching, refetching, optimistic updates
Zustand	Frontend	Lightweight client-side UI state
React Hook Form	Frontend	Form state handling

Library / Tool	Layer	Use Case
Zod	Frontend / Shared contracts	Form and payload validation, schema validation
Recharts	Frontend	Dashboard charts and KPI visualizations
Axios	Frontend	Standard HTTP client for API calls
Lucide React	Frontend	Enterprise UI iconography
Framer Motion	Frontend	High-end UI animations and transitions
SheetJS	Frontend	Client-side spreadsheet processing
jsPDF / html2canvas	Frontend	On-the-fly report generation
Transformers.js	Frontend	Local NLP and sentiment analysis

Backend libraries

Library / Tool	Layer	Use Case
NestJS v11	Backend	Main backend modular monolith framework
TypeScript	Backend	Product services, APIs, queues, business logic
Prisma ORM	Backend	PostgreSQL access, migrations, typed queries
Zod	Backend	Runtime validation of requests and event payloads
BullMQ	Backend	Queueing, async processing, inter-module events
Redis	Backend infra	Queue backing store, caching, temporary operational state
Supabase Auth SDK / JWT tools	Backend auth	Authentication and guarded access
Axios	Backend	HTTP client for service-to-service calls
Socket.io	Backend	Real-time events and streaming
Multer	Backend	File and audio upload handling
Nodemailer	Backend	Programmatic email delivery
Bcrypt	Backend	Secure credential hashing
Swagger or OpenAPI tooling	Backend docs	API discoverability and contract documentation

AI and ML libraries

Library / Tool	Layer	Use Case
Python 3.12	AI/ML	All AI and ML service implementation
FastAPI	AI/ML	Internal AI APIs such as summarize, embed, transcribe, answer-query
LiteLLM	AI/ML	LLM provider routing and fallback handling
LangGraph	AI/ML	Stateful multi-step AI workflows and agent orchestration
spaCy	AI/ML	NLP preprocessing, tokenization, entity extraction
Whisper	AI/ML	Primary speech-to-text transcription
AssemblyAI	AI/ML	Backup ASR and diarization support
pyannote.audio	AI/ML	Speaker diarization

Library / Tool	Layer	Use Case
sentence-transformers	AI/ML	Local embeddings and semantic similarity workflows
scikit-learn	AI/ML	Baseline ML, preprocessing, structured predictive models
XGBoost / CatBoost	AI/ML	Forecasting and tabular prediction
SHAP	AI/ML	Model explainability and risk factor analysis
ARIMA	AI/ML	Statistical time-series forecasting
CrewAI	AI/ML	Multi-agent autonomous workflows
PyTorch	AI/ML	Deep learning framework foundation
SQLAlchemy / Alembic	AI/ML	Python DB access and migrations
Pydantic	AI/ML	FastAPI data validation and schema enforcement
Pandas / NumPy	AI/ML	Data science and preprocessing utilities
OpenAI API client libraries	AI/ML	Managed LLM inference through Python services only

Data and search libraries / tools

Library / Tool	Layer	Use Case
PostgreSQL 16	Data	Primary relational database
pgvector	Data	Vector storage for embeddings and semantic retrieval
Meilisearch	Search	Full-text and conversation search
ClickHouse	Analytics	High-volume analytical queries and dashboard metrics
Supabase PostgreSQL	Data platform	Managed Postgres with RLS and PITR in early phases

DevOps and observability tools

Library / Tool	Layer	Use Case
Docker	DevOps	Containerization for all services
Docker Compose	DevOps	Local multi-service development
GitHub Actions	DevOps	CI/CD pipelines and quality gates
Railway	Hosting	Phase 1–2 hosting
AWS ECS / Fargate	Hosting	Phase 3 scale-up hosting
Doppler	DevOps	Secrets management
Cloudflare	Edge / Security	CDN, WAF, TLS, DDoS protection, rate limiting
Sentry	Observability	Error tracking
Better Stack	Observability	Logs and uptime monitoring
Grafana	Observability	Metrics dashboards and alerts

Testing libraries

Library / Tool	Layer	Use Case
Jest	Testing	Unit testing for TypeScript services and utilities
pytest	Testing	Unit testing for Python AI services
Supertest	Testing	Backend API integration testing

Library / Tool	Layer	Use Case
Playwright	Testing	End-to-end browser testing
Testcontainers	Testing	Real dependency integration tests
k6	Testing	Load and performance testing

Integration SDKs and APIs

Library / Tool	Layer	Use Case
Salesforce APIs / SDKs	Integration	CRM sync and enrichment
HubSpot APIs / SDKs	Integration	CRM sync and enrichment
Dynamics 365 APIs / SDKs	Integration	CRM sync and enrichment
Gmail API	Integration	Email and calendar sync, delegated sending
Outlook / Office 365 API	Integration	Email and calendar sync, delegated sending
Zoom webhooks / APIs	Integration	Meeting ingestion and recording metadata
Google Meet integrations	Integration	Meeting ingestion
Microsoft Teams webhooks / APIs	Integration	Meeting ingestion
Slack API / webhooks	Integration	Notifications and reminders
LinkedIn Sales Navigator APIs	Integration	Prospect and sales context enrichment
Clearbit / Bombora	Integration	B2B enrichment and intent data
SendGrid SDK	Integration	Transactional email delivery
Gong / Chorus	Integration	Conversation intelligence data source
HubSpot API	Integration	CRM sync and deal insertion
Slack Bolt SDK	Integration	Slack bot and app interactions
Google Calendar SDK	Integration	Calendar event synchronization

Important architecture note

- **No AI npm libraries in TypeScript product services.** The architecture explicitly forbids putting AI inference logic into the TypeScript product layer. AI and ML belong only in Python services.
- **No direct cross-module DB writes.** Modules must communicate through public APIs or events, not by touching another module's private schema.
- **No custom SMTP or owned email server logic.** Email must go through delegated Gmail or Outlook APIs only.

23. Change Log

Use this table to track updates to this document section by section. The core SAD already defines a controlled document update process, mandatory Tech Lead approval, version updates, and team notification after merge.

Version	Date	Updated By	Changes
1.0	2026-04-22	Tech Lead / AI Assistant Draft Support	Initial draft covering tools, stack, risks, libraries, approval status, payment planning, infrastructure, observability, integrations, and testing based on current SAD
1.1	YYYY-MM-DD	Name	Add or update approved tools, stack changes, or phase changes

Version	Date	Updated By	Changes
1.2	YYYY-MM-DD	Name	Update pricing, upgrade triggers, risks, or new integration decisions

24. Executive Summary

R-Revenue Intelligence is designed as an AI-powered revenue intelligence platform that captures customer interactions from calls, meetings, emails, and CRM systems, converts them into structured data through the Revenue Graph, applies AI to generate insights, and surfaces those insights into sales execution, forecasting, and coaching workflows. The architecture is intentionally built as a **modular monolith in Phase 1 and Phase 2**, with **microservice extraction only in Phase 3** after strict trigger conditions are met.

The core architecture decision is driven by the product's commercial promise: customers must be able to license only the modules they need, which means every module must have clear ownership, clean APIs, event contracts, and no illegal cross-module data access. Even though all modules deploy together today, the logical boundaries must already behave like future services.

The platform follows a **7-stage Revenue Intelligence Lifecycle: Capture → Model → Understand → Analyze → Execute → Predict → Optimize**. Each downstream stage depends on real outputs from the upstream stage, so the system cannot safely skip dependency order. This is why **M-01 Data Ingestion** is live first, **M-03 Revenue Graph** is the highest-priority next step, and many later modules remain planned until upstream events are proven end-to-end in production.

The approved language model for the engineering organization is simple and strict: **TypeScript** is used for all product services and user-facing application logic, while **Python** is used for all AI and ML services. AI inference, NLP pipelines, embeddings, ASR, diarization, and prompt workflows belong only in Python services. Business logic, APIs, auth, queue orchestration, CRM sync, and module workflows belong only in TypeScript services.

The approved frontend stack is based on **Next.js 15**, **TypeScript**, **Tailwind CSS**, **shadcn/ui**, **TanStack Query**, **Zustand**, and **React Hook Form**, giving the team a fast and standardized UI platform. The approved backend stack is based on **NestJS**, **Prisma**, **PostgreSQL**, **BullMQ**, **Redis**, and **Supabase Auth**, with strong validation and tenancy enforcement built into the platform core.

The AI and data stack centers on **FastAPI**, **LiteLLM**, **OpenAI API**, **Whisper**, **AssemblyAI**, **spaCy**, **LangGraph**, **pgvector**, **Meilisearch**, and **ClickHouse**. These tools support transcription, diarization, semantic detection, summaries, RAG, search, analytics, and structured AI workflows, while keeping AI workloads isolated from the product layer.

The infrastructure approach is optimized for speed now and scale later. **Docker** and **Docker Compose** are the local development standard, **GitHub Actions** handles CI/CD, **Railway** is the approved hosting platform for Phase 1 and 2, and **AWS ECS/Fargate** is the approved upgrade target for Phase 3. Supporting infrastructure includes **Doppler** for secrets, **Cloudflare** for CDN/WAF/rate limiting, **Sentry** for error tracking, **Better Stack** for logs and uptime, and **Grafana** for metrics and alerting.

Security and isolation are treated as non-negotiable platform rules. Tenant separation is enforced using **tenant_id**, **RLS**, Prisma-level controls, JWT guards, RBAC, and request interceptors. Webhooks are considered mission-critical entry points and must use **HMAC-SHA256 verification**, idempotency protection, Cloudflare rate limiting, and queue-based ingestion. Sensitive data such as raw audio has retention controls, and client data must never be used for shared model training without explicit written consent.

The system integrates with external CRMs, conferencing platforms, email providers, messaging tools, and client-owned data warehouses. The core approved integrations include **Salesforce**, **HubSpot**, **Microsoft Dynamics 365**, **Zoom**, **Google Meet**, **Microsoft Teams**, **Gmail**, **Outlook/Office 365**, **Slack**, **LinkedIn Sales Navigator**, **Snowflake**, **BigQuery**, **Databricks**, **Amazon S3**, and **Amazon Redshift**. The platform reads from these systems, enriches them with AI-generated outputs where allowed, and exports data back out without claiming ownership of the external system itself.

Testing is layered across **unit, integration, end-to-end, performance, container-backed dependency testing**, and **AI evaluation**. The recommended stack includes **Jest, pytest, Supertest, Playwright, Testcontainers**, and **k6**, with AI evaluation based on golden datasets, confidence scores, and human review loops. The document makes it clear that passing unit tests alone is not enough; event-flow verification across upstream and downstream modules is mandatory before calling a feature production-ready.

Operationally, the biggest current reality is that only early-stage modules are truly deployable today. **M-01 Data Ingestion** and cross-cutting compliance controls are live, while **M-03 Revenue Graph** is the next essential platform milestone. Many higher-value modules such as Smart Tracking, Deal Boards, Forecasting, Dashboards, and Coaching depend on M-03 and later event flows, so they must not be sold or promised until those dependencies are deployed and passing production data successfully.

The document also establishes governance rules for tools, upgrades, and changes. Every new shared tool or platform-level change must go through Tech Lead review, the architecture document must be updated for shared decisions, and document reviews happen every three months or immediately after major architecture changes. This ensures the system stays understandable for freshers, scalable for senior engineers, and commercially aligned for the business.

Detailed summary table

Area	What the document defines	Current approved choice	Current status	Main dependency / rule	Why it matters
Product purpose	AI revenue intelligence platform for capture, analysis, execution, forecasting, and coaching	Revenue intelligence platform built around conversation capture and AI processing	Active architecture baseline	Must support 7-stage lifecycle	Keeps all teams aligned on what the product is and is not
Architecture style	System-wide structural pattern	Modular Monolith in Phase 1–2, Microservices extraction in Phase 3	Approved	Extract only when all 5 triggers are met	Avoids premature complexity while preserving future scale
Commercial model	Why architecture must stay modular	Module-based licensing starting from low-cost entry pricing	Core constraint	No tight cross-module coupling	Directly affects pricing and packaging viability
Lifecycle model	Data and value flow across the platform	Capture → Model → Understand → Analyze → Execute → Predict → Optimize	Approved	Downstream modules require upstream outputs	Prevents teams from building features without data readiness
Module model	Logical packaging of platform features	10 modules M-01 to M-10	Approved	Modules own schema, APIs, and events	Enables future extraction and clean ownership

Area	What the document defines	Current approved choice	Current status	Main dependency / rule	Why it matters
Live module path	What can actually run today	M-01 deployed; M-03 is next critical step	Partial rollout	M-03 required for most downstream features	Protects delivery planning and sales promises
Frontend stack	UI framework and client state approach	Next.js 15, TypeScript, Tailwind CSS, shadcn/ui, TanStack Query, Zustand, React Hook Form	Approved and in use	Frontend should not contain business logic	Standardizes UI delivery and onboarding
Backend stack	Product service foundation	NestJS, TypeScript, Prisma, PostgreSQL, BullMQ, Redis, Supabase Auth	Approved and in use	Business logic stays in TypeScript only	Gives strong typing, modularity, and queue orchestration
AI/ML stack	AI inference and processing layer	Python, FastAPI, LiteLLM, OpenAI API, Whisper, AssemblyAI, spaCy, LangGraph	Approved	AI must live in Python services only	Keeps AI complexity isolated from product services
Data layer	Main transactional and retrieval stores	PostgreSQL + RLS + pgvector, Meilisearch, ClickHouse	Approved; some parts planned by phase	Source of truth stays in PostgreSQL	Supports transactions, search, analytics, and semantic retrieval
Tenant isolation	Multi-tenant security model	Shared PostgreSQL with tenant_id and RLS	Mandatory	Every write must include tenant_id and every table needs RLS	Prevents cross-tenant data leakage
Event model	Inter-module communication pattern	BullMQ + Redis event-driven communication	Approved	No illegal direct module imports or DB shortcuts	Preserves modular boundaries and async scalability
Critical events	Minimum cross-module event chain	call.transcription.completed, revenue_graph.entity.linked, tracker.detection.created, deal.stage.changed and others	Some live, some planned	Event contracts owned by publisher	Event flow is the backbone of downstream features
Hosting	Deployment platform by phase	Railway now, AWS ECS/Fargate later	Railway current, AWS planned	Migrate only when scale triggers are hit	Balances delivery speed with future control
Local development	How engineers run the system	Docker + Docker Compose	Approved and in use	Local should mirror	Makes onboarding easier and

Area	What the document defines	Current approved choice	Current status	Main dependency / rule	Why it matters
				service boundaries	reduces machine-specific issues
CI/CD	Automated validation and delivery	GitHub Actions	Approved and in use	PR checks required before merge	Keeps quality gates consistent across the team
Secrets management	Secure config storage	Doppler	Approved and in use	No secrets in code or images	Reduces credential leakage risk
Edge and protection	Internet-facing protection layer	Cloudflare	Approved and in use	Used for WAF, TLS, and rate limiting	Critical for webhook and public endpoint security
Observability	Error, log, uptime, and metrics stack	Sentry, Better Stack, Grafana	Approved and in use	Must monitor integrations and failures	Needed for production debugging and operational trust
Security controls	Platform-wide protection rules	JWT auth, RBAC, RLS, HMAC webhook validation, audit rules, data retention	Mandatory	Defense in depth, not single-layer security	Protects customer data and platform reliability
External integrations	Systems connected to the platform	Salesforce, HubSpot, Dynamics 365, Zoom, Meet, Teams, Gmail, Outlook, Slack, LinkedIn, Snowflake, BigQuery and others	Mix of deployed and planned by phase	Use approved auth and idempotency patterns	These integrations create the platform's real business value
External AI dependencies	Third-party AI services the product relies on	OpenAI API, Whisper, AssemblyAI	Approved and active	Must be called only from Python services	These power core AI features and cost center tracking
Testing strategy	Full quality strategy across layers	Jest, pytest, Supertest, Playwright, Testcontainers, k6 + AI evaluation methods	Approved / recommended	Unit tests are not enough; event-flow testing required	Ensures both correctness and production readiness
AI quality control	How AI outputs are validated	Structured JSON, confidence score, review gating, golden datasets, prompt versioning	Required for safe AI rollout	Low-confidence outputs	Improves trust and reduces bad automation

Area	What the document defines	Current approved choice	Current status	Main dependency / rule	Why it matters
				should be reviewed	
Tool governance	How tools are approved or rejected	Approved / In Use / Planned / Optional / Rejected model	Governance process defined	Tech Lead approval required for shared tools	Prevents tool sprawl and inconsistent engineering practices
Payment planning	How upgrade decisions are made	Usage-based and trigger-based upgrades across infra, AI, observability, and search tools	Planned and controlled	Upgrade only when speed, reliability, scale, or security justifies it	Helps control burn while staying production-ready
Risks	Main architectural and operational risks	Vendor lock-in, API costs, free-tier limits, privacy risks, migration risks, security risks	Active and documented	Risks must be monitored per phase	Prevents naive architecture and budget decisions
Data ownership	Who owns customer data	Client owns data and can export it; no training without consent	Mandatory rule	Data Cloud exports to client-owned destinations	Critical for trust, compliance, and enterprise readiness
Change governance	How the document and platform evolve	PR-based updates, Tech Lead approval, 3-month review cadence	Mandatory	Update SAD for any shared architecture change	Keeps the architecture as the single source of truth

Final summary note

In simple terms, this document says the team should build **one clean modular platform now**, keep **all boundaries disciplined from day one**, use **TypeScript for product logic**, use **Python for AI**, protect **tenant data very aggressively**, avoid **premature microservices**, and only unlock downstream modules when upstream events are working with real production data. That is the safest way to let freshers build fast without breaking the long-term architecture.